



J2EE Design Patterns Applied

Real World Development with Pattern Frameworks

Craig A. Berry, John Camell, Matjaz B. Juric, Meeraj Moidoo Kunnumpurath,
Nadia Nashi, Sasha Romanosky



Wrox technical support at: support@wrox.com

Updates and source code at: www.wrox.com

Peer discussion at: p2p.wrox.com

What you need to use this book

To run the samples in this book, you will need to have the following:

- ❑ A J2EE 1.3 server implementation. All the code in this book was tested on the Sun J2EE 1.3 Reference Implementation.
- ❑ A relational database. All the code in this book was tested on MySQL.

The book assumes that you are familiar with the development and deployment of J2EE components such as servlets, JSP pages and EJBs.

Summary of Contents

Introduction	1
Chapter 1: Design Patterns Applied to J2EE	7
Chapter 2: Patterns Applied to the Web Tier	41
Chapter 3: Patterns Applied to a Persistence Framework	89
Chapter 4: Patterns Applied to Improve Performance and Scalability	155
Chapter 5: Patterns Applied to Manage Security	199
Chapter 6: Patterns Applied to Enable Enterprise Integration	237
Chapter 7: Patterns Applied to Enable Reusability, Maintainability, and Extensibility	309
Index	345

3

Patterns Applied to a Persistence Framework

Data is the lifeblood of the modern corporation. Over the last five to ten years, many corporations have become keenly aware that they can no longer function as viable entities without the data contained within their IT (Information Technology) systems. Data is no longer considered a byproduct of their day-to-day business operations, but rather a strategic asset. One might even go as far as saying:

An organization's data is in many cases the organization. Without a consistent and well-defined set of services for accessing the often heterogeneous data sources within an organization, building and integrating applications can become a nightmare.

Unfortunately, even with the increased awareness of the importance of an organization's data, IT departments do not always have a well thought out strategy for how their applications access and manipulate data.

IT departments tend to look at data access as an application-level issue. They do not look at their data usage from a holistic, enterprise view. Thus, they miss opportunities to build a common framework for accessing and manipulating data locked within their corporate data sources.

Ask most software developers if their applications were written on a base development framework that can easily be modified and reused across applications and you are likely to meet one of the two types:

- ❑ The developers who have never been exposed to any kind of formal software architecture or design. Therefore, they do not know how to respond to your inquiry.
- ❑ The IT veterans who have tried to implement a framework, but found them too cumbersome to use and even more difficult to maintain. They believe frameworks would significantly reduce the complexity of their application development and maintenance efforts. However, they are rarely given the time or money to properly build a persistence framework and in the end design a solution that is not usable.

The sad reality is that most applications are not architected. They are "coded" on the fly with the tactical focus of delivering some piece of business functionality. They are not constructed with an eye towards the strategic goal of building a common set of software services that can be reused across multiple applications.

In the short term, the developer delivers what is required of them, but the organization that they work for loses out because of the long-term costs associated with supporting code that was never written with reusability or extensibility in mind.

Many IT departments are realizing that the only way they will be able to keep up with the ever-increasing demand for new applications, while trying to keep costs under control, is to build their applications around a framework that provides a common set of reusable services.

For an IT organization to properly design, deliver, and maintain any type of application framework requires:

- ❑ **Time and money**
Most organizations are not willing to appropriate time or spend any significant dollar amount to build a reusable framework. System architecture is one of the first sacrifices made by a development team when facing the (often relentless) time-to-market pressures placed on them by the different business units in their organization.
- ❑ **Incremental implementation**
Building system architecture requires identifying small deliverables that can deliver quantifiable results. Unfortunately, most companies try to implement a common system architecture using a "big bang" approach. However, they are often disappointed because they can rarely show any immediate business benefit for the several months they have to put forward in writing their initial framework.
- ❑ **Willing to change – Architectures are not designed; they evolve**
This means that there has to be willingness for an organization to refactor and redesign code, even if that code is serving its immediate purpose. Sometimes, entire pieces of architecture must be refactored to achieve the level of reuse and extensibility desired by the organization.

Many IT development teams have looked to the n-tier application model as a means of developing such a services-based framework. An n-tier application model splits an application into three logical tiers. Pieces of each tier might be physically located on different pieces of hardware throughout the enterprise. The three logical tiers in n-tier architecture are:

- ❑ A presentation tier – This is responsible for all logic that deals with presenting information to an end user.
- ❑ A business tier – This captures an organization's business rules as components that can be reused across multiple applications.
- ❑ A data tier – The data tier, also known as the integration tier, is responsible for abstracting away data sources so that the business rules can access and manipulate data regardless of where the data is located or how it is retrieved.

There has been a great deal of research done into the best practices of building a solid presentation and business tier. A walk into any bookstore will reveal that the topic of building presentation and business-tier logic has been going on over the last six to seven years.

In addition, there is a wide-range of open source presentation and business-tier frameworks available for ready use in application development projects. Two examples of these frameworks can be found on the Apache Software Foundation's Jakarta web site (<http://jakarta.apache.org>):

- ❑ Struts – Struts is a Model-View-Controller framework based on Sun's JSP Model-2 implementation. It offers a number of application plug-in points that cleanly separate an application's presentation, business, and data tiers into well-defined, maintainable pieces.
- ❑ JetSpeed – JetSpeed is an enterprise-portal development framework. JetSpeed provides a common presentation framework to quickly pull together web-based applications into a cohesive entry point for the enterprise's end users.

Frameworks like these are the result of a significant amount of focus being placed on how to cleanly separate the presentation and business-tier logic from one another.

However, until recently, few companies had thought of building their data tier using any kind of formalized, data-persistence framework. Most application developers focused exclusively on the business logic reuse in the middle tier. For many companies the data tier was a relational database or a corporate mainframe system that did not map cleanly to many of the object-oriented paradigms they were trying to implement.

The data access logic for an n-tier application is mixed with the business rules in the middle tier or vaguely defined in an amorphous set of stored procedures that attempts to hide the complexity of the data access from the developer.

However, more and more individuals are beginning to realize that failure to abstract away the physical and vendor-implemented details of their corporate data sources translates directly into a significant cost for maintaining an application long term. Some of these costs will be covered in the next section, "*What is a Persistence Framework?*"

This chapter is going to focus exclusively on developing the data tier using a combination of common data-persistence design patterns and J2EE technologies. Specifically, this chapter is going to cover the following topics:

- ❑ What is a persistence framework?
- ❑ Design patterns that can be used in the implementation of a persistence framework. The patterns covered will include:

- ❑ The Data Access Object pattern
- ❑ The Value Object pattern
- ❑ The Service Locator pattern
- ❑ Core strategies to consider when building a persistence framework:
 - ❑ Primary key generation
 - ❑ Concurrency management and locking
 - ❑ Transaction management
 - ❑ Performance issues

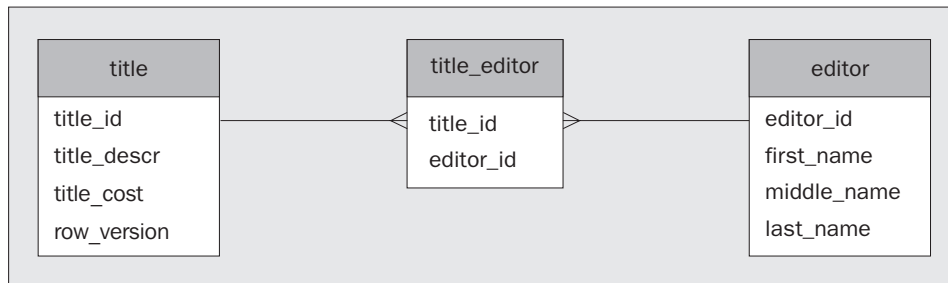
Remember, implementing a framework is often a matter of opinion. The design patterns presented in this chapter are just guidelines. In other words, they are example solutions to counter programming problems that occur repeatedly. The presentation shown in this chapter is not the only way of implementing these patterns. The key thing to remember with all data-persistence patterns is:

"Data-persistence patterns focus on building a logical façade that hides all of the ugly data access code associated with retrieving and manipulating data. They completely abstract away the physical details of the data sources that a development team uses to build its applications."

Let's say you work in the IT department of the fictional magazine publishing company, IT World Now. Your team has built and deployed several Java, web-based applications that work at a functional level. However, the management of the IT department is growing increasingly frustrated with the cost of maintaining the existing applications. They are also frustrated with the lack of reuse of the existing code base when a new application is written. They want a common application development framework that will provide reuse at all the three tiers of the application.

You have been asked to look at different aspects of IT World Now's subscription database and also at how a persistence framework can be built on top of these databases to abstract away their underlying details. You have decided to take a few data entities from the subscription database and build a prototype on which the rest of the subscription persistence framework can be built.

The prototype would be built using three tables:



- ❑ `title` – This table is used to hold all of the titles currently in the subscription database.
- ❑ `editor` – This table is used to hold all the editors currently working for IT World Now.
- ❑ `title_editor` – There is a many-to-many relationship between titles and editors. Editors can work across multiple titles and a journal can have multiple editors. The `title_editor` table establishes links between the editors and the titles that they are working on.

The SQL DDL for the tables shown above is available for download at <http://www.wrox.com/>.

A Starting Model

The following JSP page highlights some of the more common mistakes made by developers when writing an application. This JSP page is going to query the subscription database for all of the titles in the database. It will then display the results of the query on the screen. Let's take a look at the code:

```
<%@page import="java.sql.*"%>
<%@page import="java.util.*"%>

<!--
    The following JSP page demonstrates a few of the more common mistakes
    made by application developers.
-->

<table border="1" align="center">
  <tr>
    <td align="center" nowrap>
      Id
    </td>
    <td align="center" wrap>
      Description
    </td>
    <td align="center" nowrap>
      Cost
    </td>
  </tr>
</table>

<%
    Connection conn = null;
    ResultSet rs = null;
    Statement statement = null;
    try{
        /* Problem # 1 */
        Class.forName("org.gjt.mm.mysql.Driver");

        /* Problem #2 */
        conn = DriverManager.getConnection
            ("jdbc:mysql://192.168.1.98:3306/test/?username=gnu@192.168.1.98");

        /*Do something with the connection*/
        StringBuffer sql = new StringBuffer(64);
        sql.append("SELECT");
    }
}
```

```
sql.append(" title_id      ,          ");
sql.append(" title_descr ,          ");
sql.append(" title_cost   ");
sql.append("FROM          ");
sql.append(" title        ");

statement = conn.createStatement();

rs = statement.executeQuery(sql.toString());

while (rs.next()) {
    out.println("<tr>");
    out.println("<td>");
    out.println( new Long( rs.getLong("title_id") ).toString() );
    out.println("</td>");

    out.println("<td>");
    out.println( rs.getString("title_descr") );
    out.println("</td>");

    out.println("<td>");
    out.println( new Float( rs.getFloat("title_cost") ).toString() );
    out.println("</td>");
    out.println("</tr>");
}
} catch(SQLException e) {
    out.println("SQL Exception Error--> " + e.getMessage());
}
}
finally {
    if (statement!=null) try{ statement.close(); } catch(SQLException e){}
    if (rs!=null) try{ rs.close(); } catch(SQLException e){}
    if (conn!=null) try{ conn.close(); } catch(SQLException e){}
}
%>
</table>
```

This example was chosen because it demonstrates some of the typical mistakes made by database developers. These mistakes include:

- ❑ Not cleanly separating the presentation, business, and data logic into distinct tiers
- ❑ Exposing the application to what database platform the data is residing in
- ❑ Tying the application code directly to the physical structure of the database tables (in other words the `title` table)
- ❑ Forcing the application developer to learn and understand the complexity of the data access API used to retrieve and manipulate data

Separating Application Tiers

The first bullet point, not cleanly separating the presentation, business, and data logic into distinct tiers, is a still surprisingly common problem. Even with the score of n-tier architecture books and ready-to-use open source frameworks many developers fall into the trap of not separating their data logic from the other types of logic in their application:


```

while (rs.next()) {
    out.println("<tr>");
    out.println("<td>");
    out.println( new Long( rs.getLong("title_id") ).toString() );
    out.println("</td>");

    out.println("<td>");
    out.println( rs.getString("title_descr") );
    out.println("</td>");

    out.println("<td>");
    out.println( new Float( rs.getFloat("title_cost") ).toString() );
    out.println("</td>");
    out.println("</tr>");
}

```

The JSP page example is lacking any business logic, but it does demonstrate a clear lack of separation between the presentation and data tiers. Failure to enforce a clean separation of data logic from presentation and business logic will result in database code being scattered throughout the application. For small application, this might not be an issue, but for large applications this can turn in a maintenance nightmare. Even small database changes can result in developers trudging through application code looking for business code.

Hiding the Database Platform

The JSP example demonstrates the second bullet in an extreme manner. In the JSP page above, the developer has directly embedded connection information inside of the JSP page:

```

/* Problem # 1 */
Class.forName("org.gjt.mm.mysql.Driver");

/* Problem #2 */
conn = DriverManager.getConnection
("jdbc:mysql://192.168.1.98:3306/test/?username=gnu@192.168.1.98");

```

The JSP page has direct knowledge that it is talking to a MySQL database via the Java package and the class name of the MySQL JDBC driver is passed into the `Class.forName()` method call. In addition, the database connection information and user ID and password have been embedded inside of the code.

One might consider that the two problems highlighted above are minor, but they are common mistakes that often lead to a significant amount of overheads in maintaining code. Imagine a large application written in the style shown above. The application has 40 to 50 screens and any number of scenarios could force you to have to revisit each one of the screens and make changes to the database connection information. Revisiting these screens will require a significant amount of "grunt" programming work and extensive re-testing of the entire application to ensure that all database touch points were visited and changed. Some examples of the events that could trigger this include:

- ❑ Your database needs to be moved to a different server with a new IP address and machine name.
- ❑ You decide to change the database platforms that the application is working on. Your application might be using an open source database such as MySQL and you decide to move to the Oracle DBMS platform.

- ❑ The developer has compromised database security. Any one who can get access to the source code for the application now also has access to the database the application is accessing. This is especially true for JSP pages, because many developers do not pre-compile their code and leave the JSP source code directly on the service to be compiled by their application server.
- ❑ You have discovered that the pool manager you are using for your database connections has a serious flaw. You decide to switch to another vendor's pool manager.

Most developers have, finally, learned not to embed their database connection information directly inside their application code. They usually encapsulate the database connection process into some variant of a `JDBC Connection` factory class that instantiates the `JDBC Connection` for them. However, the code segment illustrates a common theme. Developers often tightly couple their application code with the database platform from which it is retrieving data. They create this tight coupling by not hiding the database connection information or using vendor-specific database features directly in their application code.

For example, the Oracle database gives the developer a database object to generate sequential numbers. This sequence and its use in the SQL syntax is Oracle specific. If the development team uses this object throughout their application, they could face a significant amount of downtime in porting the application to another database platform, as they have to search the entire application and change the code to use some other sequence generation mechanism.

Hiding the Physical Structure of your Data Entities

In the JSP example above, the developer is exposing the application to the physical structure of the data entities they are accessing. In our example, the JSP page is retrieving data from the `title` table by issuing a SQL statement directly inside of the JSP page.

```
sql.append("SELECT                ");
sql.append("  title_id           ");
sql.append("  title_descr       ");
sql.append("  title_cost        ");
sql.append("FROM                  ");
sql.append("  title                      ");
```

By issuing the SQL code directly inside of the JSP page, the developers have effectively tied themselves to the physical structure of the `title` table. If any of the columns they use are changed or deleted from the `title` table, the developers will need to visit every application that touches the `title` table and ensure that they have not broken their code.

By exposing the physical format of your data entities to the application, there is also a risk that code will break if the data relationships between the `title` table and other tables in the `subscription` database change. The SQL statement above is retrieving data just from the `title` table. However, later in the chapter you will see that there is a many-to-many relationship between the `title` and `editor` tables. If the database were refactored to make this relationship a one-to-many relationship, again all of the applications that issued any SQL commands against the `subscription` database would have to be revisited.

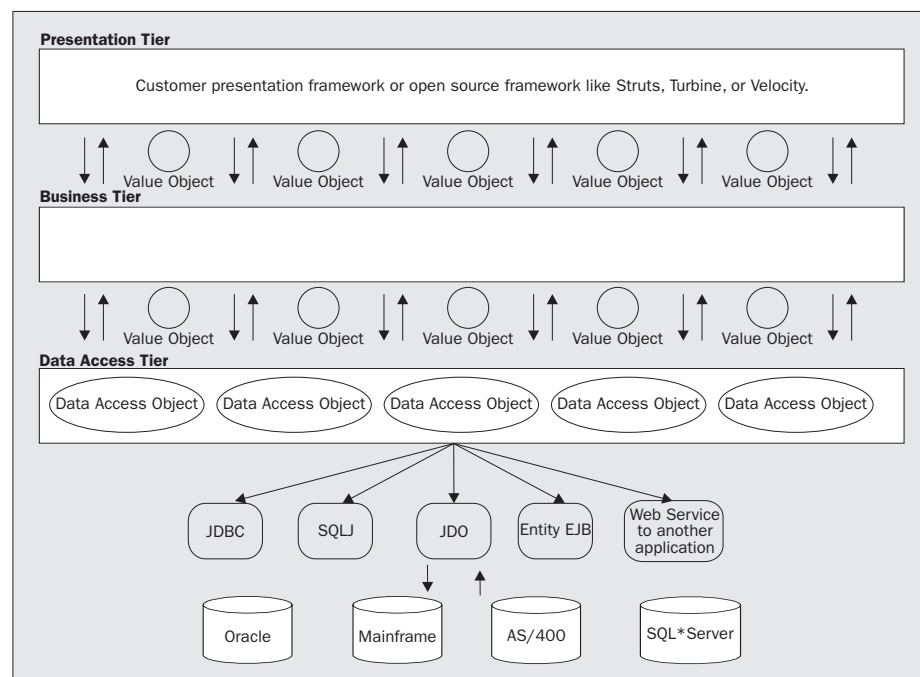
Encapsulating Complexity

The last point to take away from this code example is that every time the developer wants to retrieve or manipulate any data from the subscription database, they must understand the complexities of a data access API, such as JDBC or JDO (Java Data Objects). The developer must understand how to connect to the database and the syntax of a query language, and properly understand such concepts as transaction management and isolation.

Most IT organizations want to divide their application developers into tiers. They want their novice and intermediate-level developers to focus on building business applications and leave the infrastructure and architectural code to more experienced and seasoned developers. The only way these organizations can accomplish this is by ensuring that complex and often repetitive tasks like accessing data from a corporate database are hidden from the business application developers. These developers should not have to know where or how they are getting their data. Business application developers should focus on writing business code and not infrastructure code.

What is a Persistence Framework?

A persistence framework is a set of software services that decouples an application from the data sources it uses and manipulates. A persistence framework sits on top of an organization's data sources and hides the data access APIs (such as JDBC, JDO, or entity EJBs) used to access these data sources. The services offered should completely abstract away the physical details of accessing and manipulating data from that data source. The diagram below illustrates the where in the system architecture the persistence framework sits:



In the above diagram, the data persistence framework completely abstracts away how data is being accessed. It does this by strictly enforcing the separation of the presentation, business, and data tiers. To aid in this separation, two design patterns are used:

- ❑ The Data Access Object pattern
- ❑ The Value Object pattern

The Data Access Object pattern completely encapsulates the retrieving and manipulating of data. It also encapsulates the data access API used to interact with the database such as JDBC, JDO, SQLJ, and so on. The Data Access Object pattern communicates with the business tier by using value objects that encapsulate data being retrieved and sent to the database.

The Value Object pattern shown in the diagram encapsulates data sent to and from the database. The Value Object pattern hides the physical table structures from the business application developers and abstracts away what data relationships exist in the database, by providing a Java Collection mechanism for retrieving parent-child, one-to-many, and many-to-many relationships from the database. The value objects can also be passed up to the presentation tier and used in displaying information to the end users.

In large organizations, you might have dozens of systems that are using different mechanisms for data access. In addition, the organization can have data that is being accessed by application developers scattered throughout systems in the organization that are running on a plethora of database platforms. Why expose these differences to the developers trying to write business applications? A persistence framework hides these complexities from an application developer writing a business application.

The services offered in a persistence framework should do the following for the developers building applications on it:

- ❑ **Provide a clean mechanism that encourages the separation of data-persistence logic from presentation and business logic**
In the architectural diagram above, the business tier is the only tier allowed to access corporate databases. The business tier interacts with corporate databases by using the Data Access Object pattern. It never uses a data access API like JDBC to access a database.
- ❑ **Decouple the applications built on the framework from having any knowledge of the database platform the data is residing in**
By encapsulating all data connection logic inside the data access objects, the business application developer does not need to know what type of database platform they are connecting to, any of the security information (user ID and password) needed to connect to the database, or the network address of the database.
- ❑ **Abstract away the physical details of how data is stored within a database and also the relationships that might exist between data entities in the database**
Applications built on the architecture layout above never issue SQL queries directly against a database. They have no true idea of what the physical structure of the data is and use value objects to access the database.
- ❑ **Simplify the development process by hiding the details associated with opening a connection to a database, issuing commands to retrieve and manipulate data, and transaction management**
The Data Access Object and Value Object patterns completely decouple the business application developer from the data source they are using in their application. They do not need knowledge of a data access API and instead can use simple interfaces for retrieving and manipulating data.

Let's now dive into the first of the two patterns mentioned above, the Data Access Object pattern, and explore how it can be implemented.

The Data Access Object Pattern

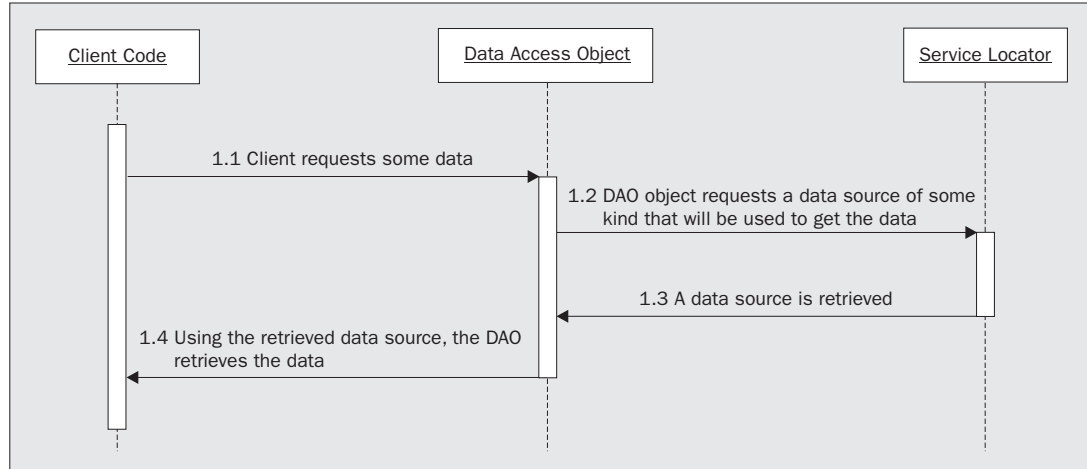
The Data Access Object (DAO) and Value Object (VO) patterns were first documented at the Sun Java Center. The DAO pattern abstracts away how data is retrieved and manipulated from a data source.

The DAO pattern serves two purposes:

- ❑ First, the DAO pattern completely abstracts away the data source in which the data the user is requesting resides. The user of the DAO pattern does not know whether the data they are retrieving is coming from an Oracle database, a Microsoft SQL Server, an LDAP server, or a web service. The data source is completely transparent.
- ❑ The second function the DAO pattern provides is that it abstracts away the CRUD (Create, Replace, Update, and Delete) logic normally associated with accessing a data source. As the user has no knowledge of how the data is being queried or retrieved, they will not be affected if the data access code (for example, a SQL statement issued inside the DAO) is changed or modified. A DAO pattern even allows an application architect to pull data from multiple data sources and present it to the user as one logical object (in other words a value object containing data from multiple data sources).

This flexibility provided by the DAO pattern comes from the fact that applications do not directly access a data source. Instead, they create a DAO object and use it to access a data source on their behalf. If they are retrieving data, they can use a value object to hold the retrieved data. The Value Object pattern abstracts away the physical structure of how the data is stored in the database. The Value Object pattern will be discussed in greater detail later in the chapter.

The diagram below illustrates conceptually how a piece of client code uses a DAO object to retrieve data on its behalf:



The DAO pattern is powerful because it:

- ❑ Abstracts away how data is actually being retrieved from a data source. A DAO pattern can use any number of Java data access APIs to carry out its work. Some of these APIs are:
 - ❑ The JDBC API – The first standard database access API, JDBC is part of the J2SE with extensions in J2EE.
 - ❑ SQLJ – SQLJ is relatively new API put forth by several leading database and Java application-server vendors. SQLJ allows the developer to embed SQL code directly into their Java code and then have it translated into Java code using a code preprocessor.
 - ❑ Entity Enterprise Java Beans – The entity EJB specification is a data-persistence specification that allows the developer to abstract away database code behind a component-based architecture.
 - ❑ Java Data Objects (JDO) – The JDO specification is newest addition to database access APIs. JDO allows you to retrieve and persist data from a database using standard Java objects. JDO objects are lighter weight than entity EJBs, while still providing much of the same functionality as an entity EJB.
- ❑ Provides a clean separation of business and data logic to minimize the temptation for application developers to mix their business and data logic. By centralizing all of the data access code within one logical construct, developers can use a logical interface for accessing and manipulating data.
- ❑ Provides a standard set of programming interfaces that will address the common database development issues of:
 - ❑ Primary Key Generation
 - ❑ Concurrency Management and Locking
 - ❑ Transaction Management
 - ❑ Performance Issues

Persistence Framework Exceptions

All exceptions both caught from and raised within the persistence framework will be rethrown as a `DataAccessException`. This rethrowing is being implemented for two reasons:

- ❑ By catching all exceptions thrown within the data persistence framework, there is an opportunity to process and catch all exceptions in a consistent manner. Developers using data-persistence framework classes like the `DataSourceFactory` need to worry about catching only one exception.
- ❑ By rethrowing any exceptions raised within the `DataSourceFactory`, or any other part of our data-persistence framework, we are able to abstract away the data sources that the persistence framework accesses. The persistence framework developer provides a consistent exception that can always be caught by the application code using the framework.

The code for the `DataAccessException` is shown below:

```
package dataaccess;

public class DataAccessException extends Exception {
    Throwable exceptionCause = null;

    public DataAccessException(String pExceptionMsg) {
        super(pExceptionMsg);
    }

    public DataAccessException(String pExceptionMsg, Throwable pException) {
        super(pExceptionMsg);
        this.exceptionCause = pException;
    }

    public void printStackTrace() {
        if (exceptionCause != null) {
            System.err.println("An exception has been caused by: ");
            exceptionCause.printStackTrace();
        }
    }
}
```

The `DataAccessException` class is a generalized exception. It is used to rethrow any non-persistence framework exceptions raised and any minor issues that might arise during processing within the persistence framework. Other more specialized exceptions within the framework reflect specific events that might want to be tracked and caught by the application using the DAO. These exceptions all extend the `DataAccessException` and include:

- ☐ `NoDataFoundException`
- ☐ `OptimisticLockException`
- ☐ `ServiceLocatorException`

These exceptions will be covered as they are encountered in our walkthrough of the persistence framework.

Implementing the Data Access Object Pattern

The DAO pattern abstracts away the tasks associated with retrieving and manipulating data by defining a Java interface that has five basic methods:

- ☐ `findByPrimaryKey(Object pPrimaryKey)`
- ☐ `insert(ValueObject pValueObject)`
- ☐ `createValueObject()`
- ☐ `update(ValueObject pValueObject)`
- ☐ `delete(ValueObject pValueObject)`

The `findByPrimaryKey()` method is used to retrieve a record from a data source. It takes a single input parameter, `pPrimaryKey`. This parameter is the unique ID of the record in the data source. In the examples for this chapter, this parameter represents a primary key field for a record in a relational database. The `findByPrimaryKey()` method returns an object of type `ValueObject`. A `ValueObject`, which is a wrapper, wraps the data retrieved from the data source the DAO represents as a Java class. Thus, the code using the DAO is never tied directly to the underlying data structure that the DAO encapsulates.

The `insert()`, `update()`, and `delete()` methods all carry out the actions that their names imply. They take a single parameter of type `ValueObject` that holds all the data that will be used in carrying out the insert, update, or delete actions.

You might wonder why on the `delete()` method you are passing in an entire `ValueObject` rather than just the primary key ID. The main reason is efficiency. If the data entity represented by the DAO has several conditions that must be met in order for the record to be deleted, it is easier to try to pass that data into the DAO as part of the `ValueObject` passed into the method. Otherwise, for each condition that must be checked before deleting the record, the DAO must perform a database lookup to confirm that condition has been met.

This condition often exists when you are building your DAO layer above a database written by a third-party vendor. As the persistence framework developer, you do not have control over how these vendors support referential integrity for their data entities when data is deleted. Many vendors embed their referential integrity code directly inside their application. This means you have to reverse engineer the database behavior and put this behavior in your DAO to ensure that the proper insert, updates, and deletes occur when a record is deleted from the database.

The `createValueObject()` method is used to create an empty `ValueObject` that can be populated with data and then passed to a DAO's `insert()` method for insertion into the data source represented by the DAO.

Since a DAO pattern abstracts the mechanism by which data is retrieved and manipulated, there are two different ways to implement it:

- ❑ As a standard Java class
- ❑ As a J2EE session EJB

All of the DAOs built in this chapter will be built as stateless session EJBs using JDBC. The DAO could also have been built using a simple Java class.

Simple Java classes are quicker to implement and take less time to debug because there is significantly less setup work to write a class. However, the J2EE stateless session EJB approach has been chosen for this chapter because the application server that the EJB runs in provides a number of core infrastructure services. These infrastructure services include:

- ❑ **Container-based transaction management**
This relieves the database developer of the responsibility of having to code complex transaction management code within their application.
- ❑ **Declarative Access Control**
Declarative access control allows the developer to control what users and application can use the session EJB to carry out their work.

❑ **Resource pooling**

Resource pooling improves application performance by keeping a pool of session EJBs available to process user requests.

In order to build a framework for EJB-based DAOs, we are going to extend the EJB remote interface (EJBObject) and the bean class (SessionBean) to include the five DAO method signatures defined earlier.

The DAO Remote Interface

The DAOObject interface will be used as the remote interface for our data access objects:

```
package dataaccess;

import javax.ejb.EJBObject;
import java.rmi.RemoteException;
import valueobject.ValueObject;

public interface DAOObject extends EJBObject {

    public ValueObject findByPrimaryKey(Object pPrimaryKey)
        throws DataAccessException, NoDataFoundException,
        ServiceLocatorException, RemoteException;

    public void insert(ValueObject pValueObject)
        throws DataAccessException, ServiceLocatorException,
        RemoteException;

    public void update(ValueObject pValueObject)
        throws DataAccessException, OptimisticLockException,
        ServiceLocatorException, RemoteException;

    public void delete(ValueObject pValueObject)
        throws DataAccessException, ServiceLocatorException,
        RemoteException;

    public ValueObject createValueObject()
        throws DataAccessException, ServiceLocatorException,
        RemoteException;

}
```

The DAOBean Class

The DAOBean class extends the SessionBean interface used in a standard session EJB implementation:

```
package dataaccess;

import javax.ejb.SessionBean;
import valueobject.ValueObject;

public interface DAOBean extends SessionBean {
```

```
public ValueObject findByPrimaryKey(Object pPrimaryKey)
    throws DataAccessException, NoDataFoundException,
        ServiceLocatorException;

public void insert(ValueObject pValueObject)
    throws DataAccessException, ServiceLocatorException;

public void update(ValueObject pValueObject)
    throws DataAccessException, OptimisticLockException,
        ServiceLocatorException;

public void delete(ValueObject pValueObject)
    throws DataAccessException, ServiceLocatorException;

public ValueObject createValueObject()
    throws DataAccessException, ServiceLocatorException;
}
```

The TitleDAO Session Bean

We are going to build a DAO object to represent the `title` table in the subscription database. Since this DAO is implemented as a session EJB, it will consist of three Java files:

- ❑ `TitleDAO`
- ❑ `TitleDAOHome`
- ❑ `TitleDAOBean`

Firstly, the `TitleDAO` interface is the remote interface for the EJB. It extends the `DAOObject` interface as shown below:

```
package session.title;

import dataaccess.DAOObject;

public interface TitleDAO extends DAOObject {}
```

Next, the `TitleDAOHome` interface extends the `EJBHome` interface. This interface is responsible for returning the `TitleDAOBean` class:

```
package session.title;

import javax.ejb.EJBHome;
import java.rmi.RemoteException;
import javax.ejb.CreateException;

public interface TitleDAOHome extends EJBHome {

    public TitleDAO create() throws RemoteException, CreateException;

}
```

The TitleDAOBean Class

The `TitleDAOBean` class holds the actual implementation of the `TitleDAO` code. This class is a wrapper around all of the database code that is normally exposed and intermixed within presentation and business logic.

As you are looking through the `TitleDAOBean` implementation, it is important to keep these two key points in mind:

- ❑ The `TitleDAO` implementation shields the developer from how data is being retrieved and manipulated. It could easily be converted from JDBC to some other data access API.
- ❑ The physical structure of the `title` table can be easily restructured. The table can easily be normalized or denormalized.

Both of these tasks can be accomplished because the `TitleDAO` provides a public interface that completely hides the underlying database code. Radically change your data structures too much and even a DAO might not be able to keep the change from rippling through all of the applications in your organization. However, a DAO provides a buffer that will allow the application architecture to isolate applications from most changes made to a database. Without the abstraction layer that a DAO object provides, even small changes to the database can represent significant code rewrites and testing in multiple applications throughout the enterprise.

Shown below is the skeleton for the `TitleDAOBean` class (each method will be walked through in greater detail in its own section):

```
package session.title;

import dataaccess.*;
import valueobject.*;
import session.counter.*;
import session.editor.*;
import javax.ejb.*;
import java.sql.*;
import java.rmi.RemoteException;
import java.util.HashMap;
import java.util.Collection;
import java.util.Iterator;

public class TitleDAOBean implements DAOBean {
    private SessionContext ctx;
    /*
     * The findByPrimaryKey() method will retrieve a single title
     * record from the subscription database. It returns a TitleVO
     * that contains all of the title data.
     */
    public ValueObject findByPrimaryKey(Object pPrimaryKey)
        throws DataAccessException, ServiceLocatorException,
            NoDataFoundException {
    }
}
```

```
/*
 * The update() method will update a single record in the title
 * table with the valueobject passed in as a parameter.
 */
public void update(ValueObject pValueObject)
    throws DataAccessException, OptimisticLockException,
           ServiceLocatorException {
}

/*
 * The delete() record will delete a single record from the title
 * table in the subscription database. It does this based on the
 * id of the value object passed into the table.
 */
public void delete(ValueObject pValueObject)
    throws DataAccessException, ServiceLocatorException {
}

/*
 * The insert() method will insert a single record into the title
 * table. The data it inserts will be pulled from the value
 * object passed into the method.
 */
public void insert(ValueObject pValueObject)
    throws DataAccessException, ServiceLocatorException {
}

/*
 * The createValueObject() method will create an empty value
 * object prepopulated with any mandatory data. n example of
 * this mandatory data might be a primary key that for a particular
 * record.
 */
public ValueObject createValueObject()
    throws DataAccessException, ServiceLocatorException {
}

public void ejbCreate() {}
public void ejbRemove() {}
public void ejbActivate() {}
public void ejbPassivate() {}
public void setSessionContext(SessionContext sessionCtx) {
    this.ctx = sessionCtx;
}
}
```

The *findByPrimaryKey()* Method

The `findByPrimaryKey()` method is used to retrieve a single record from the `title` table. It returns the data in the form of a `ValueObject` object. We'll see the implementation of this later in the chapter.

The code for the `TitleDAO.findByPrimaryKey()` method uses JDBC calls to retrieve the requested record from the subscription database and create a `ValueObject`, populate it with data, and return it to the application calling the `find()` method:

```
public ValueObject findByPrimaryKey(Object pPrimaryKey)
    throws DataAccessException, ServiceLocatorException,
        NoDataFoundException {

    String pkPrimaryKey = (String) pPrimaryKey;
    ServiceLocator serviceLocator = ServiceLocator.getInstance();
    Connection conn =
        serviceLocator.getDBConn(ServiceLocator.SUBSCRIPTIONDB);
    ResultSet rs = null;

    //Getting my SQL Code for retrieving all of the titles
    SQLCode sqlCode = SQLCode.getInstance();
    String sql =
        sqlCode.getSQLStatement("titledao.findbyprimarykey().select");

    TitleVO titleVO = new TitleVO();

    try {
        PreparedStatement preparedStatement = conn.prepareStatement(sql);
        preparedStatement.setLong(1, Long.parseLong(pkPrimaryKey));
        rs = preparedStatement.executeQuery();

        // Populate a the value object with the resultset data
        if (rs.next()){
            titleVO.setTitleId(rs.getLong("title_id"));
            titleVO.setTitleDescr(rs.getString("title_descr"));
            titleVO.setTitleCost(rs.getFloat("title_cost"));
            titleVO.setRowVersion(rs.getLong("row_version"));

            EditorDAOHome editorDAOHome =
                (EditorDAOHome)serviceLocator.getEJBHome(
                    ServiceLocator.EDITORDAO);
            EditorDAO editorDAO = editorDAOHome.create();
            titleVO.setEditors(
                editorDAO.findEditorByTitle(String.valueOf(
                    titleVO.getTitleId())));
        } else {
            throw new NoDataFoundException(
                "Record id: " + pkPrimaryKey +
                "is not found in the title table.");
        }

        //Reset the flags so that we know the object is in pristine state.
        titleVO.resetFlags();
    } catch(SQLException e) {
        //Aborting the transaction
        ctx.setRollbackOnly();
        throw new DataAccessException(
```

```
        "Error in TitleDAO.findByPrimaryKey()", e);
    } catch (RemoteException e) {
        //Aborting the transaction
        ctx.setRollbackOnly();
        throw new DataAccessException(
            "Error in TitleDAO.findByPrimaryKey()", e);
    } catch (CreateException e) {
        //Aborting the transaction
        ctx.setRollbackOnly();
        throw new DataAccessException(
            "Error in TitleDAO.findByPrimaryKey()", e);
    } finally {
        try {
            if (rs != null) rs.close();
            if (conn != null) conn.close();
        } catch (SQLException e) {
            System.out.println("I am unable to close a resultset, " +
                               "statement, or connection " +
                               "TitleDAO.findByPrimaryKey().");
        }
        return titleVO;
    }
}
```

The first several lines of code of the `findByPrimaryKey()` method may look unusual. The first thing the `findByPrimaryKey()` method does is cast the primary key argument to a `String`. This is because this is the data type we are going to be using in our application, but you could just as easily cast it to any object type.

Then the method uses the `ServiceLocator` object to retrieve the `JDBC` Connection to the subscription database and `EJBHome` interfaces for all `EJBs` used within the persistence framework:

```
ServiceLocator serviceLocator = ServiceLocator.getInstance();

Connection conn =
    serviceLocator.getDBConn(ServiceLocator.SUBSCRIPTIONDB);
```

The `ServiceLocator.getDBConn()` method retrieves a `JDBC` database Connection to the subscription database. We will be covering the `ServiceLocator`'s code later in the chapter.

After the database connection is retrieved, we use an object called `SQLCode` to retrieve the `SELECT` statement used to retrieve data from the `title` table:

```
SQLCode sqlCode = SQLCode.getInstance();
String sql =
    sqlCode.getSQLStatement("titledao.findbyprimarykey().select");
```

The `SQLCode` class is used to hold all of the `SQL` statements used in the persistence framework in one location. A `DAO` needing a `SQL` statement can get an instance of the `SQLCode` class and then issue a call to the `SQLCode.getSQLStatement()` method. When this method is called, a key value is passed in that will map to the desired `SQL` statement. The `SQL` statement used in the `TitleDAO.findByPrimaryKey()` method looks like:

```
SELECT title_id, title_descr, title_cost, row_version FROM title WHERE
       title_id=?
```

The `SQLCode` class is based on the Singleton design pattern and has two purposes:

- ❑ Centralize all of the SQL statements used in the application within a single property file called `sql.properties`. Using the `SQLCode` class, all changes to the SQL code in the persistence framework take place in this one location.
- ❑ Eliminate the need to build the SQL statement directly inside of each of the `find()`, `insert()`, `update()`, and `delete()` methods in a DAO. The `SQLCode` class has only one static copy of a particular SQL statement. This way if a particular DAO method is called a dozen times, there is only one `String` object representation of that SQL statement stored within in the virtual machine.

The `SQLCode` class is shown below. However, it is very straightforward Java code and will not be covered in greater detail in this chapter.

```
package dataaccess;

import java.util.Properties;
import java.io.*;

/*
 * The SQLCode class is a helper class that loads all of the SQL
 * code used within the persistence framework into a private properties
 * object stored within the SQLCode class.
 */
public class SQLCode {
    private static SQLCode sqlCode = null;
    private static Properties sqlCache = new Properties();

    /*Calling the SQLCode's constructor*/
    static {
        sqlCode = new SQLCode();
    }

    /*Retrieves the Singleton for the SQLCode class*/
    public static SQLCode getInstance() {
        return sqlCode;
    }

    /*
     * The SQLCode constructor loads all of the SQL code used in the
     * persistence framework from the sql.properties class into the sqlCache
     * properties object
     */
    private SQLCode() {
        try {
            String sqlFileName = "sql.properties";

            sqlCache.load(new FileInputStream(sqlFileName));
        } catch (IOException e) {
            // Handle exception
        }
    }
}
```

```
        } catch(IOException e) {
            System.out.println(e.toString());
        }
    }

    /**
     * The getSQLStatement() method will try to retrieve a SQL statement
     * from the SQLCache properties object based on the key passed into
     * the method. If it can not find an the SQL statement, a DataAccess
     * Exception will be raised.
     */
    public String getSQLStatement(String pSQLKeyName)
        throws DataAccessException {

        //Checking to see if the requested SQL statement is in the sqlCache
        if (sqlCache.containsKey(pSQLKeyName)) {
            return (String) sqlCache.get(pSQLKeyName);
        } else{
            throw new DataAccessException("Unable to locate the SQL " +
                "statement requested in SQLCode.getSQLCode() " + pSQLKeyName);
        }
    }
}
```

After the abstractions offered by the `ServiceLocator` and `SQLCode` classes, the rest of the `findByPrimaryKey()` method executes the SQL statement and populates a `TitleVO` object with the data retrieved from the title table:

```
PreparedStatement preparedStatement = conn.prepareStatement(sql);
preparedStatement.setLong(1, Long.parseLong(pPrimaryKey));
rs = preparedStatement.executeQuery();

if (rs.next()){
    titleVO.setTitleId(rs.getLong("title_id"));
    ...
}
```

There are a few things to keep in mind about the `findByPrimaryKey()` method.

First, the base DAO interface defines a single `findByPrimaryKey()` method to retrieve a record by its primary key. However, there is nothing to stop a DAO developer from implementing multiple `find()` methods. Each implemented `find()` method on a DAO can retrieve data via different queries. If multiple records are retrieved from a query, the `find()` method should return a `HashMap` containing `ValueObjects`. Each `ValueObject` in the `HashMap` represents a single record retrieved from the query encapsulated within the `find()` method.

In the `TitleDAO.findByPrimaryKey()` method, the `EditorDAO.findEditorByTitle()` method is used to retrieve all of the editors assigned to that particular title.

For the sake of brevity, the `EditorDAO` code will not be shown in this chapter. However, as mentioned earlier, the code is available for download from the Wrox web site.

This `findEditorByTitle()` method returns a `HashMap` containing all of the editors associated with that title. Why use a `HashMap` to hold the results of multiple records? Why not use an `ArrayList`? The `HashMap` object was chosen because it supports retrieving a record from within it by using a key value. In the `TitleDAO.findByPrimaryKey()` method, the `HashMap` being returned by the call to `EditorDAO.findEditorByTitle()` is keyed with the primary key of each editor.

If we know the editor's ID, we can retrieve the record directly from the `HashMap` without iterating through every item in the map. If we used a `Collection` object like an `ArrayList` or `Vector`, we would not have this flexibility in locating a record in the collection. The `HashMap` returned by the `EditorDAO.findByPrimaryKey()` method is then passed into the `setEditors()` method on the `titleVO` object:

```
EditorDAOHome editorDAOHome =
    (EditorDAOHome) new InitialContext().lookup("editor/EditorDAO");
EditorDAO editorDAO = editorDAOHome.create();

titleVO.setEditors(editorDAO.findEditorByTitle
    (new Long(titleVO.getTitleId()).toString()));
```

This `TitleDAO.findByPrimaryKey()` method demonstrates how our persistence framework can handle a one-to-many or many-to-many database relationship. A many-to-many relationship exists between the `title` table and the `editor` table. When a user tries to retrieve a `TitleVO`, the `TitleDAO.findByPrimaryKey()` method will also retrieve all of the editor records associated with that particular title and capture that relationship in the `TitleVO` object passed back to the end user.

If no data is found by the `findByPrimaryKey()` method, it will raise a `NoDataFoundException`. This exception inherits from the `DataAccessException` as described. This exception does not necessarily indicate that there is an error in the DAO, just that a record could not be retrieved.

There is one thing to be aware of with using the `EditorDAO.findEditorByTitle()` method to retrieve all of the editors associated with a title: this method call can become a performance bottleneck. For example, if you are retrieving 1000 title records, there will be 1000 calls to the `EditorDAO.findEditorByTitle()` method. This is a fundamental weakness inherent to a persistence framework.

Persistence frameworks are designed to handle small transactions that retrieve a few objects, make changes and update them. They are not meant for retrieving large data sets or performing bulk inserts, updates, or deletes. In the `findEditorByTitle()` case, if you believe that you are going to bring back a large dataset, you might want to evaluate whether or not you want to use the design patterns shown in this chapter for that data work.

The insert() Method

The code for the `insert()` method is shown below. Let's walk through the `insert()` method and discuss what is occurring within it. The first step in the `insert()` method is to retrieve the SQL code that is going to be used in the `TitleDAO.insert()` method:

```
public void insert(ValueObject pValueObject)
    throws DataAccessException, ServiceLocatorException {

    //Getting all of my SQL code for inserting a title
```

```
SQLCode sqlCode = SQLCode.getInstance();

String sqlTitle = sqlCode.getSQLStatement("titledao.insert().insert");
String sqlTitleEditor =
    sqlCode.getSQLStatement("titledao.insert().inserttitleeditor");
```

The `insert()` method is going to use two SQL statements while executing. The first SQL statement will insert the title data from the `TitleVO` passed into the `insert()` method into the title table. This SQL statement, represented by the key `titledao.insert().insert`, looks like:

```
INSERT INTO title (title_id, title_descr, title_cost, row_version)
VALUES ( ?,?,?,? )
```

Once the SQL code is retrieved, a database connection is obtained from a `ServiceLocator`. The retrieved database connection is then used to obtain a `PreparedStatement`:

```
//Building my SQL Code for inserting a record into the title table.
TitleVO titleVO = (TitleVO) pValueObject;
ServiceLocator serviceLocator = ServiceLocator.getInstance();
Connection conn =
    serviceLocator.getDBConn(ServiceLocator.SUBSCRIPTIONDB);

PreparedStatement preparedStatement = null;

try {
    //Populate the prepared statement with data from the value object
    preparedStatement = conn.prepareStatement(sqlTitle);
```

Note also in the above code, that the `ValueObject` passed into the `insert()` method is cast to type `TitleVO`. The `TitleVO` instance, called `titleVO`, is used to populate the `preparedStatement` object. Once the `preparedStatement` object has been populated, it executes the `INSERT` statement:

```
preparedStatement.setLong(1, titleVO.getTitleId());
preparedStatement.setString(2, titleVO.getTitleDescr());
preparedStatement.setFloat(3, titleVO.getTitleCost());
preparedStatement.setLong(4,1);
preparedStatement.execute();
```

After the record has been inserted into the title table, there needs to be a relationship established with the editor table. The `insert()` method will retrieve a `HashMap` containing all the editor records that are to be linked with the title record:

```
HashMap editorHash = titleVO.getEditors();
Collection editorCol = editorHash.values();
Iterator editorIterator = editorCol.iterator();
```

It will then walk through each of the EditorVO objects in the editorIterator object, adding an entry in the title_editor table as it goes:

```
while (editorIterator.hasNext()) {
    EditorVO editorVO = (EditorVO) editorIterator.next();
    preparedStatement = conn.prepareStatement(sqlTitleEditor);
    preparedStatement.setLong(1, titleVO.getTitleId());
    preparedStatement.setLong(2, editorVO.getEditorId());
    preparedStatement.execute();
}
} catch (SQLException e) {
    //Aborting the transaction
    ctx.setRollbackOnly();
    throw new DataAccessException(
        "Error in TitleDAO.findByPrimaryKey()", e);
} catch (RemoteException e) {
    //Aborting the transaction
    ctx.setRollbackOnly();
    throw new DataAccessException(
        "Error in TitleDAO.findByPrimaryKey()", e);
} catch (CreateException e) {
    //Aborting the transaction
    ctx.setRollbackOnly();
    throw new DataAccessException(
        "Error in TitleDAO.findByPrimaryKey()", e);
} finally {
    try {
        if (rs != null) rs.close();
        if (conn != null) conn.close();
    } catch (SQLException e) {
        System.out.println("I am unable to close a resultset, " +
            "statement, or connection " +
            "TitleDAO.findByPrimaryKey().");
    }
    return titleVO;
}
}
```

If everything works successfully, the finally{} block of the code is executed and the preparedStatement and resulting database connection are closed. If any exception is encountered during the course of the insert, like an SQLException, then it will be caught and re-wrapped as a DataAccessException.

Looking at the code for the insert() method, you might have two questions:

- ❑ How and where is the primary key for the record generated?
- ❑ How is the transaction being committed to the database?

The primary key for a record in the title table is the title_id. We see that the value inserted for this code comes from the getTitleId() method on the titleVO object. How the actual primary key value is generated will be discussed in the *Primary Key Generation Strategies* section later on in this chapter.

A database commit is not taking place in the following code. This is because the TitleDAO session EJB is using container-based transaction management. The application server is handling all data commits. The only thing the code worries about is notifying the application when an exception has been raised and the transaction has to be rolled back. This is done by the `setRollbackMethod()` on the `ctx` object. The `ctx` object is the `SessionContext` for the TitleDAO EJB:

```
ctx.setRollbackOnly();
```

The whole issue of transaction management will be covered in the *Transaction Management Strategies* section of this chapter.

The `createValueObject()` Method

The `createValueObject()` method found in our DAO implementation is used to create a `ValueObject` that can be populated with data and then passed to the `insert()` method of the DAO from which it was created. The `createValueObject()` method acts in many ways like a constructor. It gives the DAO developer the opportunity to properly initialize a `ValueObject` before it is inserted into a data source.

Remember, in *The `insert()` Method* section above, a question was raised as to where a primary key was generated for a record. The answer is in the `createValueObject()` method. Here is the `createValueObject()` implementation for the TitleDAO:

```
public ValueObject createValueObject()
    throws DataAccessException, ServiceLocatorException {

    TitleVO titleVO = new TitleVO();

    try {
        //Creating a counter instance. We will use this counter instance
        //to retrieve a primary key for the new record being used.
        ServiceLocator serviceLocator = ServiceLocator.getInstance();

        CounterHome counterHome = (CounterHome)
            serviceLocator.getEJBHome(ServiceLocator.COUNTER);
        Counter counter = counterHome.create();
        titleVO.setTitleId(counter.getNextVal("title_pk"));

        //Populating the rest of the value object with data
        titleVO.setTitleDescr("");
        titleVO.setTitleCost(0);
        titleVO.setRowVersion(1);
        titleVO.resetFlags();
    } catch(CreateException e){
        //Aborting the transaction
        ctx.setRollbackOnly();
        throw new DataAccessException(
            "Error in TitleDAO.createValueObject()", e);
    } catch(RemoteException e){
        //Aborting the transaction
        ctx.setRollbackOnly();
    }
```

```

        throw new DataAccessException(
            "Error in TitleDAO.createValueObject()", e);
    } finally {
        return titleVO;
    }
}

```

In the code above, a new TitleVO object is created and prepopulated with data. The interesting part of the code shows how the primary key is generated for the new TitleVO value object.

The primary key for the ValueObject is being generated by creating a session EJB called Counter and calling its getNextVal() method:

```

ServiceLocator serviceLocator = ServiceLocator.getInstance();

CounterHome counterHome = (CounterHome)
    serviceLocator.getEJBHome(ServiceLocator.COUNTER);
Counter counter = counterHome.create();
titleVO.setTitleId(counter.getNextVal("title_pk"));

```

The Counter EJB, which will be covered in greater detail in the *Primary Key Generation Strategies* section, is used to generate all of the primary keys for the subscription database. The createValueObject() method has a completely database-independent mechanism for generating primary keys. This is important because almost every database platform available has its own mechanism for generating primary keys. The Counter EJB decouples how the primary key generation is carried out from the rest of the application and persistence-framework code.

Move your application to another database platform and you only have to change the primary key generation for your applications in one location, rather than several. This simple abstraction can save a significant amount of work if you ever move your DAOs from one database platform to another.

The createValueObject() method has a database-independent mechanism for generating primary keys. The Counter EJB that generates these primary keys can still very much be database dependent.

The update() Method

The update() method for the TitleDAO is shown below. This method updates the title table with the data contained within the ValueObject passed into it:

```

public void update(ValueObject pValueObject)
    throws DataAccessException, OptimisticLockException,
        ServiceLocatorException {

    //Getting my SQL Code for updating all of the titles
    SQLCode sqlCode = SQLCode.getInstance();
    String sql = sqlCode.getSQLStatement("titledao.update().update");

    TitleVO titleVO = (TitleVO) pValueObject;
}

```

```

ServiceLocator serviceLocator = ServiceLocator.getInstance();
Connection conn =
    serviceLocator.getDBConn(ServiceLocator.SUBSCRIPTIONDB);

PreparedStatement preparedStatement = null;

try {
    //Retrieving a row version number via the Counter session bean
    CounterHome counterHome =
        (CounterHome) serviceLocator.getEJBHome(ServiceLocator.COUNTER);
    Counter counter = counterHome.create();
    long nextRowVersion = counter.getNextVal("title_rv");

    //Populating the prepared statement's parameters
    preparedStatement = conn.prepareStatement(sql);
    preparedStatement.setString(1, titleVO.getTitleDescr());
    preparedStatement.setFloat( 2, titleVO.getTitleCost());
    preparedStatement.setLong( 3, nextRowVersion);
    preparedStatement.setLong( 4, titleVO.getTitleId());
    preparedStatement.setLong( 5, titleVO.getRowVersion());

    //Check to see if we were successful in updating the record.
    // If the queryResult does not equal 1, then we have run
    // into an optimistic lock situation.
    int queryResults = preparedStatement.executeUpdate();
    if (queryResults != 1) {
        ctx.setRollbackOnly();
        throw new OptimisticLockException("Stale data for title record: "
            + titleVO.getTitleId());
    }
    //Cyclr through all of the editors and seeing if an editor
    //needs to be updated.
    Collection col = titleVO.getEditors().values();
    Iterator iterator = col.iterator();

    EditorDAOHome editorDAOHome = (EditorDAOHome)
        serviceLocator.getEJBHome(ServiceLocator.EDITORDAO);
    EditorDAO editorDAO = editorDAOHome.create();

    while (iterator.hasNext()) {
        EditorVO editorVO = (EditorVO) iterator.next();
        //If the updateFlag has been set to true then update the editor
        // vo record by invoking update on the editorDAO.
        if (editorVO.getUpdateFlag()) {
            editorDAO.update(editorVO);
        }
    }
} catch(SQLException e) {
    //Aborting the transaction
    ctx.setRollbackOnly();
    throw new DataAccessException("Error in TitleDAO.update()", e);
} catch(CreateException e) {
    //Aborting the transaction

```

```

        ctx.setRollbackOnly();
        throw new DataAccessException("Error in TitleDAO.update()", e);
    } catch (RemoteException e) {
        //Aborting the transaction
        ctx.setRollbackOnly();
        throw new DataAccessException("Error in TitleDAO.update()", e);
    } finally {
        try {
            if (preparedStatement!=null) preparedStatement.close();
            if (conn!=null) conn.close();
        } catch (SQLException e) {
            System.out.println("I am unable to close a resultset, " +
                               "statement, or connection TitleDAO.update().");
        }
    }
}
}

```

The code for the `update()` method starts by getting SQL code for executing the update to the title table and retrieving a JDBC connection to the subscription database:

```

SQLCode sqlCode = SQLCode.getInstance();
String sql = sqlCode.getSQLStatement("titledao.update().update");

TitleVO titleVO = (TitleVO) pValueObject;

ServiceLocator serviceLocator = ServiceLocator.getInstance();
Connection conn =
    serviceLocator.getDBConn(ServiceLocator.SUBSCRIPTIONDB);

```

Next, the `update()` method uses the Counter EJB to retrieve the next row version number that is going to be applied when updating the targeted title table record:

```

CounterHome counterHome =
    (CounterHome) serviceLocator.getEJBHome(ServiceLocator.COUNTER);
Counter counter = counterHome.create();
long nextRowVersion = counter.getNextVal("title_rv");

```

The `update()` method uses an optimistic locking strategy to handle multiple users trying to update the same database record at the same time. An optimistic locking strategy does not apply an explicit database lock to a record when a user retrieves it from the database. Instead, an optimistic locking strategy lets each user retrieve a copy of the same record. The record is versioned and when the user tries to update the record in the database, this version number is checked to make sure that the data the user is trying to update has not already been updated by another user.

There are a number of different ways of implementing an optimistic locking strategy. For the purpose of this chapter's implementation, the optimistic locking strategy requires that every table in the database have a column called `row_version`.

The `row_version` column contains a number that indicates the version number of the row. In the data-persistence framework built in this chapter, when a record is retrieved from the database, it is retrieved into a `ValueObject`. As part of the data put into the `ValueObject`, a `row_version` is retrieved.

When a user wishes to commit any changes that they have made to a record, a `ValueObject` containing the data is passed into the `update()` method. The `update()` method will then use the `row_version` as part of the `WHERE` for the `UPDATE` statement. The `UPDATE` statement used in the `TitleDAO EJB` is shown below:

```
UPDATE title SET title_descr=?, title_cost=?, row_version=? WHERE title_id=? AND
row_version=?
```

In the `UPDATE` statement above, the `row_version` is being updated as part of the `UPDATE SQL` statement. The updating of the row's version number is used to indicate that the row has changed and other users who are using a copy of this record now have stale data. The generation of a `row_version` number for each row in the `title` table is done by the `Counter EJB`. Each table in the subscription database has its own entry in the counter table that keeps track of the current row version number for that table.

Once the row version number has been generated, a prepared statement is populated with the data from the `TitleVO` object passed in and the newly generated row version. This `PreparedStatement` is then executed:

```
preparedStatement = conn.prepareStatement(sql);
preparedStatement.setString(1, titleVO.getTitleDescr());
preparedStatement.setFloat( 2, titleVO.getTitleCost());
preparedStatement.setLong( 3, nextRowVersion);
preparedStatement.setLong( 4, titleVO.getTitleId());
preparedStatement.setLong( 5, titleVO.getRowVersion());
```

So, what happens when two users try to update the same record in the `title` table at the same time? Both users will have received the same version of the record via a `ValueObject` they retrieved from the `TitleDAO's find()` method. User one issues an update by calling the `update()` method on the `TitleDAO EJB`. User one's record will successfully update the database because the `UPDATE` statement will find a match on a record in the database. In the process of updating the record, the `row_version` number will change for the record that user one just updated.

User two's data is now stale. When this user tries to update the database via the `TitleDAO.update()` method, it will fail because the `UPDATE` statement will not be able to find a record whose `title_id` and `row_version` match that of the `ValueObject` passed into the `update()` method. The `update()` method checks to see if the update has been successfully executed by looking at the number of rows updated by the `UPDATE` statement:

```
int queryResults = preparedStatement.executeUpdate();
if (queryResults != 1) {
    throw new OptimisticLockException("Stale data for title record: "
                                     + titleVO.getTitleId());
}
```

If the number of rows returned by the `UPDATE SQL` call does not equal 1, then the user trying to update the `title` table will know they have stale data.

If an optimistic lock situation does occur, an `OptimisticLockException` is thrown. The `OptimisticLockException` is a user-defined exception that extends the `DataAccessException`. The `OptimisticLockException` is thrown to the application calling the update so it can make a decision on what to do with the data. If the application using the DAO is a user interface, the end user might be informed that someone has changed the data they are working on and that they must submit their changes again.

Different implementations of the optimistic locking strategy are discussed later in the chapter in the Concurrency Strategies section.

After the record in the title table is updated, the `update()` method will create an `EditorDAO` EJB:

```
EditorDAOHome editorDAOHome = (EditorDAOHome)
    serviceLocator.getEJBHome(ServiceLocator.EDITORDAO);
EditorDAO editorDAO = editorDAOHome.create();
```

After the `EditorDAO` EJB is created, the `update()` method will walk through each of the `EditorVO`'s objects associated with the `TitleVO`. If an `EditorVO`'s update flag is set to true, determined by calling the `EditorVO.getUpdateFlag()` method, the data for the `EditorVO` is updated by passing that `EditorVO` into the `EditorDAO.update()` method:

```
while (iterator.hasNext()) {
    EditorVO editorVO = (EditorVO) iterator.next();
    if (editorVO.getUpdateFlag()) {
        editorDAO.update(editorVO);
    }
}
```

The insert, update, and delete flags are used when a DAO is going to allow the users of the DAO to insert, update, or delete `ValueObjects` contained within the `ValueObject` passed into the DAO. In the case of `TitleDAO`, we are giving an application the ability to update not only a `TitleVO`, but also any `EditorVO`s contained within that particular `TitleVO`. The user tries to retrieve them with the `HashMap` contained with the `TitleVO`, change any values they want changed, and then set the update flag to true for that `EditorVO`. When the `update()` method is then called on the `TitleDAO`, that `EditorVO` will also be updated in the subscription database.

The delete() Method

The `delete()` method not only deletes the record from the title table, it also deletes all entries associated with the targeted title record from the `title_editor` table. This extra delete is done to ensure referential integrity between the title table and the `title_editor` table:

```
public void delete(ValueObject pValueObject)
    throws DataAccessException, ServiceLocatorException {

    //Build the SQL Code for deleting a record from the title table.
    SQLCode sqlCode = SQLCode.getInstance();
    String sqlTitle =
        sqlCode.getSQLStatement("titledao.delete().deletetitle");
    String sqlTitleEditor =
```

```

        sqlCode.getSQLStatement("titledao.delete().deletetitleeditor");

        TitleVO titleVO = (TitleVO) pValueObject;

        ServiceLocator serviceLocator = ServiceLocator.getInstance();
        Connection conn =
            serviceLocator.getDBConn(ServiceLocator.SUBSCRIPTIONDB);

        PreparedStatement preparedStatement = null;

        try {
            //Delete the row from the title table
            preparedStatement = conn.prepareStatement(sqlTitle);
            preparedStatement.setLong(1, titleVO.getTitleId());
            preparedStatement.executeQuery();

            //Delete all rows from the title_editor table that match the
            //title_id
            preparedStatement = conn.prepareStatement(sqlTitleEditor);
            preparedStatement.setLong(1, titleVO.getTitleId());
            preparedStatement.executeQuery();
        } catch (SQLException e) {
            //Aborting the transaction
            ctx.setRollbackOnly();
            throw new DataAccessException("Error in TitleDAO.delete()", e);
        } finally{
            try {
                if (preparedStatement!=null) preparedStatement.close();
                if (conn!=null) conn.close();
            } catch (SQLException e) {
                System.out.println("I am unable to close a resultset, " +
                                    "statement, or connection. TitleDAO.delete()");
            }
        }
    }
}

```

DAOs and Relationships

Capturing and modeling relationships between entities in your persistence framework can be a difficult task. In the `TitleDAO` code, there is a many-to-many relationship between the `TitleVO` and the `EditorVO`. It is up to the developer of the persistence framework to determine how much control they are going to allow applications to modify either the data entities or the data relationships that exists between items represented in the framework.

In its implementation in this chapter, the `TitleDAO` only manages the relationship between the `title` table and the `editor` table. It does not allow the developer to add new editors that are absent in the `editor` table nor delete any editors from the `editor` table by marking them for deletion (that is, setting the `deleteFlag` to true on an `EditorVO` object). The only thing that the `TitleDAO` does is to:

- Add entries to the `title_editor` table when a new title record is inserted. It will walk through the `TitleVO`'s `editors` `HashMap` and add an entry for each `EditorVO` to the `HashMap`. However, the code assumes that the editor record exists in the `editor` table already.

- ❑ Delete entries from the `title_editor` table when an existing title is deleted from the `title` table.
- ❑ Allow for updates of `EditorVO` objects contained within a `TitleVO` passed into a `TitleDAO.update()` method.

One of the core design decisions you will have to make in your persistence framework is at what level you want to give a DAO control and management of records belonging to another data entity in your database.

The management of data relationships is more than a code issue; it is a data management issue. How much control you give a DAO to add, update, and delete records outside of its immediate scope, as is the case with the `TitleDAO` being allowed to update records in the `editor` table, should be given careful consideration. Managing data relationships in code is extremely difficult. Poorly written code can corrupt your database by breaking the referential integrity that exists between data entities in the database.

The Value Object Pattern

The Value Object pattern first came into use as a business-tier J2EE design pattern that helps minimize the number of remote calls that are needed when using entity EJBs. Before the EJB 2.0 specification, any time a client made a call on an EJB's methods, a remote call would take place. This remote method call could have a significant amount of network overhead even if the client calling the EJB was located in the same application server hosting the EJB.

The Value Object pattern helped mitigate network overhead by having the entity EJB populate a plain Java class with all of the data requested by the user. The value object would then be sent back to the application calling the entity bean. The calling application could then work with the value object and then pass it back to the entity it originated from. By having the calling application getting and setting data on the value object, rather than the EJB, the amount of potential network traffic associated with using an entity EJB is significantly reduced. We'll be looking at this use of the Value Object pattern in the next chapter.

However, the Value Object pattern can be extended for significant use in our persistence framework. One could even argue that the Value Object pattern could be considered a core data-tier pattern. There are two reasons why this argument can be made:

- ❑ Firstly, a value object can be used to represent a logical view of data contained within your databases. The Value Object pattern can be used to abstract away the data relationships (such as whether or not there exists a one-to-many or many-to-many relationship between entities). The value object can even be used to present a single interface for data pulled from multiple data sources. As far as the client application using the value object is concerned, it has no idea where the data came from or how it is going to be managed.
- ❑ The second reason is that a value object is the glue that holds all tiers of n-tier architecture together. Value objects can be used to pass data back and forth across the presentation, business-logic, and data tiers. When a user keys data into a user interface, that data can be put into a value object, passed to the business-logic tier and then forwarded to the data tier. Conversely, when a user requests information, the data tier can return a value object (or a set of value objects) back up to the business-logic tier, which can then forward these value objects to the presentation tier.

All value objects in this chapter's data-persistence framework descend from a single abstract class called `ValueObject`, which is shown below:

```
package valueobject;

import java.io.Serializable;

/*
 * The ValueObject is the abstract class that all ValueObjects in our
 * data persistence framework descend off of. It provides a number of
 * methods, including get/set methods for the VO status flags and the
 * rowVersion method.
 */
public abstract class ValueObject implements Serializable {

    /*Below are the VO status flags*/
    private boolean insertFlag = false;
    private boolean updateFlag = false;
    private boolean deleteFlag = false;
    private long    rowVersion = 0;

    /*
     * Retrieves the insertFlag. The insertFlag is used to tell the DAO that
     * data in the ValueObject should be inserted into the database.
     */
    public boolean getInsertFlag() {
        return insertFlag;
    }

    /*
     * Retrieves the deleteFlag. The deleteFlag is used to tell the DAO that
     * the data in the ValueObject should be deleted from the database.
     */
    public boolean getDeleteFlag() {
        return deleteFlag;
    }

    /*
     * Retrieves the updateFlag. The updateFlag is to tell the DAO that the
     * data in the ValueObject should be updated in the database.
     */
    public boolean getUpdateFlag() {
        return updateFlag;
    }

    /*
     * Retrieves the rowVersion. The rowVersion holds the current rowVersion
     * of the record and is used by the DAO.update() method to enforce
     * optimistic locking exceptions.
     */
    public long getRowVersion() {
        return rowVersion;
    }
}
```

```

/*Sets the insertFlag and sets all other status flags to false*/
public void setInsertFlag(boolean pFlag) {
    insertFlag = pFlag;
    deleteFlag = false;
    updateFlag = false;
}

/*Sets the updateFlag and sets all other status flags to false*/
public void setUpdateFlag(boolean pFlag) {
    insertFlag = false;
    deleteFlag = false;
    updateFlag = pFlag;
}

/*Sets the deleteFlag and sets all other status flags to false*/
public void setDeleteFlag(boolean pFlag) {
    insertFlag = false;
    deleteFlag = pFlag;
    updateFlag = false;
}

/*Sets the rowVersion property*/
public void setRowVersion(long pRowVersion) {
    rowVersion = pRowVersion;
}

/*Resets all VO status flags to false*/
public void resetFlags() {
    insertFlag = false;
    deleteFlag = false;
    updateFlag = false;
}
}

```

The ValueObject class has three status properties with their own `get()` or `set()` methods:

- ☐ `insertFlag`
- ☐ `updateFlag`
- ☐ `deleteFlag`

These three flags are used to give hints as to what a DAO should do with a value object. Earlier in the chapter, we saw how the `TitleDAO` bean could update the editor table when the `TitleDAO.update()` method was called.

Remember, in the `TitleDAO.update()` method, the update flag was used to give a "hint" to the `TitleDAO` as to what it should do with `EditorVOs` contained within the `TitleVO`. The `TitleDAO` will only perform an update on an editor record when the update flag for the `EditorVO` is set.

Only one of these flags can be set at any given time. A call to any one of the set methods of these status flags will automatically set the other status flags to false. For instance, if the `setInsertFlag()` method is called, the `insertFlag` for the record is set to whatever value is passed in by the calling application. The `updateFlag` and `deleteFlag` will automatically be set to false:

```
public void setInsertFlag(boolean pFlag) {
    insertFlag = pFlag;
    deleteFlag = false;
    updateFlag = false;
}
```

The `ValueObject` class also provides a `reset()` method. This method will automatically reset the value of the insert, update, and delete status flags to be `false`.

As you will see shortly, the `TitleVO` automatically sets the `updateFlag` to `true` every time a call to any of its set methods is made. This can be problematic when a DAO is creating a `ValueObject` for the first time and populating it with data. Before the `ValueObject` can be sent back to the end user, its update status has already been set to `true`. The `reset()` method can be used to take this populated `ValueObject` and make sure all of its status flags have been set back to `false`:

```
public void resetFlags(){
    insertFlag = false;
    deleteFlag = false;
    updateFlag = false;
}
```

Finally, the `ValueObject` class provides `get()` and `set()` method for the `rowVersion` attribute. The `rowVersion` attribute was put in the `ValueObject` class to ensure that every `ValueObject` in our persistence framework participates in the framework's optimistic locking strategy.

We have seen the `ValueObject` class at abstract level. Let's look at a more concrete example of it by looking at the `TitleVO` value object:

The TitleVO Value Object

The `TitleVO` class returns all of the information associated with the `title` table. It also captures the relationship that exists between the `title` table and the `editor` table via the `getEditors()` or `setEditors()` methods. However, the `TitleVO` class does not reveal to the application using it what the relationship between the `title` and the `editor` table is. This is an important abstraction because the relationship between the `title` and the `editor` table can be changed. This change has to be updated in the `TitleDAO`, but the `TitleVO` does not have to change. This means applications consuming data from the `TitleVO` are protected from the underlying database change.

All the application code needs to know is that it can find all of the editor records associated with a title record simply by calling the `TitleVO.getEditors()` method. This is a powerful abstraction, particularly, if at some point the database relationship between the `title` and the `editor` tables needs to be restructured:

```
package valueobject;

import java.util.HashMap;

/*
 * The TitleVO class represents data retrieved from the ValueObject.
 * It is an implementation of the ValueObject pattern.
 */
```

```
*/
public class TitleVO extends ValueObject {
    private long    titleId;
    private String  titleDescr;
    private float   titleCost;
    private HashMap editors;

    /*Constructor: Initializes the properties of the class to be empty*/
    public TitleVO() {
        titleId    = 0;
        titleDescr = "";
        titleCost   = 0;
        editors     = new HashMap();
    }

    /*Retrieves the title id*/
    public long getTitleId() {
        return titleId;
    }

    /*Retrieves the title description*/
    public String getTitleDescr() {
        return titleDescr;
    }

    /*Retrieves the title cost*/
    public float getTitleCost() {
        return titleCost;
    }

    /*Returns all the editors associated with a title*/
    public HashMap getEditors() {
        return editors;
    }

    /*Sets the title id*/
    public void setTitleId(long pTitleId) {
        setUpdateFlag(true);
        titleId = pTitleId;
    }

    /*Sets the title descr*/
    public void setTitleDescr(String pTitleDescr) {
        setUpdateFlag(true);
        titleDescr = pTitleDescr;
    }

    /*Sets the title cost*/
    public void setTitleCost(float pTitleCost) {
        setUpdateFlag(true);
        titleCost = pTitleCost;
    }
}
```

```
/*Sets a hashmap with all of the editors*/  
public void setEditors(HashMap pEditors) {  
    setUpdateFlag(true);  
    editors = pEditors;  
}  
}
```

When designing your value object layer keep in mind the following:

- ❑ **Your value objects should be lightweight**
A value object is nothing more than a wrapper for data going in and out of a database. This means there should be little to no code inside your value object implementation. If you find that you are placing business logic inside your value object, consider re-factoring your value object to take the logic out. The logic is better placed in either the business tier or the database in the DAO.
- ❑ **Value objects are a view of data within your database**
There is nothing wrong with having your DAO's `find()` method return different variations or views of a value object. This is particularly true when a DAO is servicing many types of applications. For instance, one `find()` method might return a single `TitleVO` fully populated with editor information. Another `find()` method might return a `HashMap` of `TitleVOs` that do not have their editor values populated. You can even have a single DAO return completely different types of value objects.
- ❑ **Keep your value object's object hierarchy simple**
If you need to build a complex object hierarchy to return to an application, do it in the business-logic layer. Have your business object take value objects from several different DAOs and assemble them into its hierarchy. This will significantly reduce the complexity of the database code being developed in your DAOs.
- ❑ **Watch out for the number of value objects in your persistence framework**
A common mistake of developers is to make their value objects too fine grained. The result is an explosion of small value objects that become unwieldy to use and maintain.

The Service Locator Pattern

Throughout the examples in this chapter, you have seen several occasions where EJBs and JDBC database connections were retrieved for use by the persistence framework. These two items were retrieved using the JNDI (Java Naming Directory Interfaces) API. How JNDI lookups are accomplished can vary from vendor to vendor. In addition, repeated JNDI lookups can be expensive.

Each time a user needs a service retrieved from JNDI, they must populate the appropriate JNDI environment information, make a connection to the JNDI server and then look up the service. What is needed is a single point for creating and retrieving various services from JNDI. This is where the Service Locator pattern is useful.

The Service Locator pattern abstracts away all JNDI lookups behind a simple-to-use interface. The Service Locator implementation used in this chapter provides methods for retrieving `EJBHome` interface and JDBC database connections. In addition, this Service Locator minimizes the number of JNDI lookups required by caching `EJBHome` interfaces and the `DataSource` objects used to create JDBC connections the first time a user requests a specific EJB or JDBC connection.

The code for the Service Locator pattern is shown below:

```
package dataaccess;

import java.sql.*;
import javax.sql.DataSource;
import java.util.Hashtable;
import javax.naming.*;
import javax.ejb.*;
import javax.rmi.PortableRemoteObject;
import session.counter.CounterHome;
import session.title.TitleDAOHome;
import session.editor.EditorDAOHome;

/*
 * The Service Locator pattern is abstracts away the JNDI
 * logic necessary for retrieving a JDBC Connection or EJBHome
 * interface
 */
public class ServiceLocator{
    private static ServiceLocator serviceLocatorRef = null;
    private static Hashtable      ejbHomeCache      = null;
    private static Hashtable      dataSourceCache    = null;

    /*Enumerating the different services available from the
     * Service Locator
     */
    public static final int COUNTER          = 0;
    public static final int TITLEDAAO        = 1;
    public static final int EDITORDAAO       = 2;
    public static final int SUBSCRIPTIONDB   = 3;

    /*The JNDI Names used to lookup a service*/
    private static final String COUNTER_JNDINAME="counter/Counter";
    private static final String TITLEDAAO_JNDINAME="title/TitleDAO";
    private static final String EDITORDAAO_JNDINAME="editor/EditorDAO";
    private static final String
        SUBSCRIPTIONDB_JNDINAME="java:/subscriptionDS";

    /*References to each of the different EJB Home Interfaces*/
    private static final Class COUNTERCLASSREF = CounterHome.class;
    private static final Class TITLECLASSREF   = TitleDAOHome.class;
    private static final Class EDITORCLASSREF  = EditorDAOHome.class;

    static {
        serviceLocatorRef = new ServiceLocator();
    }

    /*Private Constructor for the ServiceLocator*/
    private ServiceLocator(){
        ejbHomeCache      = new Hashtable();
        dataSourceCache    = new Hashtable();
    }
}
```

```
/*
 * The ServiceLocator is implemented as a Singleton. The getInstance()
 * method will return the static reference to the ServiceLocator stored
 * inside of the ServiceLocator Class.
 */
public static ServiceLocator getInstance(){
    return serviceLocatorRef;
}

/*
 * The getServiceName will retrieve the JNDI name for a requested
 * service. The service is indicated by the ServiceId passed into
 * the method.
 */
static private String getServiceName(int pServiceId)
    throws ServiceLocatorException {

    String serviceName = null;

    switch (pServiceId) {
        case COUNTER:
            serviceName = COUNTER_JNDINAME;
            break;
        case TITLEDAAO:
            serviceName = TITLEDAAO_JNDINAME;
            break;
        case EDITORDAAO:
            serviceName = EDITORDAAO_JNDINAME;
            break;
        case SUBSCRIPTIONDB:
            serviceName = SUBSCRIPTIONDB_JNDINAME;
            break;
        default:
            throw new ServiceLocatorException(
                "Unable to locate the service requested in " +
                "ServiceLocator.getServiceName() method. ");
    }
    return serviceName;
}

/*
 * Returns the EJBHome Class reference for a requested service.
 * If the method can not make a match, it will throw a
 * ServiceLocatorException.
 */
static private Class getEJBHomeRef(int pServiceId)
    throws ServiceLocatorException {
    Class homeRef = null;

    switch (pServiceId){
        case COUNTER:
            homeRef = COUNTERCLASSREF;
            break;
        case TITLEDAAO:
```

```

        homeRef = TITLECLASSREF;
        break;
    case EDITORDAO:
        homeRef = EDITORCLASSREF;
        break;
    default:
        throw new ServiceLocatorException(
            "Unable to locate the service requested in " +
            "ServiceLocator.getEJBHomeRef() method.  ");
    }
    return homeRef;
}

/*
 * The getEJBHome method will return an EJBHome interface for a
 * requested service.  If it can not find the requested EJB, it will
 * throw a servicelocator exception.
 *
 * The getEJBHome interface caches a requested EJBHome so that the first
 * time an EJB is requested, a home interface will be retrieved but then
 * be placed into a cache.
 */
public EJBHome getEJBHome(int pServiceId)
    throws ServiceLocatorException {
    //Trying to find the JNDI Name for the requested service
    String serviceName = getServiceName(pServiceId);
    EJBHome ejbHome = null;

    try {
        //Checking to see if I can find the EJBHome interface in cache
        if (ejbHomeCache.containsKey(serviceName)) {
            ejbHome = (EJBHome) ejbHomeCache.get(serviceName);
            return ejbHome;
        } else {
            //If I could not find the EJBHome interface in the cache, look it
            // up and then cache it.
            Context ctx = new InitialContext();
            Object jndiRef = ctx.lookup(serviceName);

            Object portableObj =
                PortableRemoteObject.narrow(jndiRef, getEJBHomeRef(pServiceId));

            ejbHome = (EJBHome) portableObj;

            ejbHomeCache.put(serviceName, ejbHome);
            return ejbHome;
        }
    } catch (NamingException e) {
        throw new ServiceLocatorException(
            "Naming exception error in ServiceLocator.getEJBHome()" ,e);
    } catch (Exception e) {
        throw new ServiceLocatorException(
            "General exception in ServiceLocator.getEJBHome",e);
    }
}

```

```
}

/*
 * The getDBConn() method will create a JDBC connection for the
 * requested database. It too uses a cache algorithm to minimize
 * the number of JNDI hits that it must perform.
 */
public Connection getDBConn(int pServiceId)
    throws ServiceLocatorException {
    //Getting the JNDI Service Name
    String serviceName = getServiceName(pServiceId);
    Connection conn = null;

    try {
        //Checking to see if the requested DataSource is in the Cache
        if (dataSourceCache.containsKey(serviceName)) {
            DataSource ds = (DataSource) dataSourceCache.get(serviceName);
            conn = ((DataSource)ds).getConnection();
            return conn;
        } else {
            // The DataSource was not in the cache. Retrieve it from JNDI
            // and put it in the cache.
            Context ctx = new InitialContext();
            DataSource newDataSource = (DataSource) ctx.lookup(serviceName);
            dataSourceCache.put(serviceName, newDataSource);
            conn = newDataSource.getConnection();
            return conn;
        }
    } catch(SQLException e) {
        throw new ServiceLocatorException(
            "A SQL error has occurred in ServiceLocator.getDBConn()", e);
    } catch(NamingException e) {
        throw new ServiceLocatorException(
            "A JNDI Naming exception has occurred "+
            " in ServiceLocator.getDBConn()", e);
    } catch(Exception e) {
        throw new ServiceLocatorException(
            "An exception has occurred in ServiceLocator.getDBConn()", e);
    }
}
```

Let's start our discussion with the `getEJBHome()` method. This method is used to retrieve an `EJBHome` interface from the `ServiceLocator`:

```
public EJBHome getEJBHome(int pServiceId)
    throws ServiceLocatorException {
```

When a user wants a specific EJB, they call this method, passing in a service ID. These service IDs are defined as static final int variables in the ServiceLocator class:

```
public static final int COUNTER          = 0;
public static final int TITLEDAAO        = 1;
public static final int EDITORDAAO       = 2;
public static final int SUBSCRIPTIONDB   = 3;
```

When one of these values is passed into the getEJBHome() method it is converted to a JNDI name. The JNDI name is what is passed to a JNDI context's lookup() method to retrieve an item from the naming directory. The conversion of the service ID to the JNDI name is accomplished by calling the getServiceName() method:

```
String serviceName = getServiceName(pServiceId);
```

The getServiceName() method is nothing more than a switch/case statement that will match a JNDI name with the service ID passed into it. If a match is not found, the getServiceName() method will throw a ServiceLocatorException. This exception is raised whenever an abnormal situation occurs within the ServiceLocator.

After the JNDI name is retrieved, the cache will be checked to see if there is already an EJBHome interface that has already been retrieved from JNDI. The cache for this ServiceLocator implementation is a Hashtable that uses the JNDI name for the EJB as the key:

```
if (ejbHomeCache.containsKey(serviceName)) {
    ejbHome = (EJBHome) ejbHomeCache.get(serviceName);
    return ejbHome;
}
```

If a match is made on the cache, the EJBHome interface is retrieved out of the cache and returned to the application calling it. If the EJBHome interface is not in the cache, then a JNDI lookup will be performed:

```
} else {
    Context ctx = new InitialContext();
    Object jndiRef = ctx.lookup(serviceName);
```

The ctx.lookup() method above will return a reference to the item looked up via JNDI. The code then tries to cast the abstract interface returned from the JNDI lookup to its appropriate type. This is done by using the PortableRemoteObject.narrow() call, passing in the item to be checked with the Class type we want to cast it to:

```
Object portableObj =
    PortableRemoteObject.narrow(jndiRef, getEJBHomeRef(pServiceId));
```

The actual Class type that is going to be passed into the narrow() method will be retrieved by a call to getEJBHomeRef(). This method will retrieve a class reference for the particular EJB requested. The getEJBHomeRef(), like the getServiceName(), uses a switch/case statement to map a service name to a Class type.

Once the EJBHome has been retrieved, it is put into the cache and then returned to the application requesting it:

```
ejbHomeCache.put(serviceName, ejbHome);  
return ejbHome;
```

The `getDBConn()` method works in the same manner as the `getEJBHome()` method. This method uses JNDI to retrieve a `DataSource` object representing the database connection requested by the user. It then uses that `DataSource` to create a database connection. Like the `getEJBHome()` method, the `getDBConn()` method caches a `DataSource` object when it first retrieves it from JNDI. After the `DataSource` is cached, any requests for it will cause the `DataSource` to be pulled out of the cache rather than a new JNDI lookup.

One final thing to note about this `ServiceLocator` implementation is that it implemented as a Singleton pattern. As you have seen throughout all of the examples in the chapter, users never create a `ServiceLocator` by calling its constructor. Instead, they use the `ServiceLocator`'s `getInstance()` method to retrieve a reference to the static `ServiceLocator` stored within the `ServiceLocator` class:

```
public static ServiceLocator getInstance() {  
    return serviceLocatorRef;  
}
```

Implementing the `ServiceLocator` as a Singleton pattern is extremely efficient because the class is heavily used and does not need to maintain instance information.

The use of non-final static methods in the `ServiceLocator` is actually in violation of the EJB specification (see section 21.4.2). However, we can get away with this because we're not dependent on our cache being consistent across Java Virtual Machines, which is the reason that the restriction exists. You must also verify that your factory classes can be used by multiple threads concurrently without synchronization; this issue is not addressed in the specification. These two issues make our solution perform better but also make it slightly less portable. We could trade some performance back for portability by doing the caching in the bean instance instead.

The Persistence Framework in Action

Let's now walk through two examples of how to use the data-persistence framework we have built to retrieve and insert title data. These examples consist of two JSP pages called:

- ❑ `selectExample.jsp`
The `selectExample.jsp` shows how to retrieve a single title record from the `TitleDAO.find()` method. It will also show how to retrieve a `TitleVO`'s editor information by using the `getEditors()` method on the `TitleVO` object retrieved from the `TitleDAO`.
- ❑ `insertExample.jsp`
The `insertExample.jsp` will demonstrate how to use a `TitleDAO` to insert a record into the title table. It will also use the `EditorDAO` to retrieve `EditorVO`s that can then be associated with the title record being inserted.

You should *never* have your JSP pages directly accessing your DAO layer. Instead, your JSP pages should use a Session Faade to access the DAOs on behalf of the page. However, for purposes of demonstrating the DAO, to build this extra layer of abstraction would significantly increase the amount of code needed to demonstrate the actions of inserting and retrieving data with a DAO. In addition, the Session Faade is more relevant to the next chapter so we'll leave its discussion until then.

For more information on how to properly layer your presentation, business, and data tiers refer to the *Migration Strategies* section at the end of the chapter.

The selectExample.jsp Page

```
<%@page import="java.util.*"%>
<%@page import="javax.naming.*"%>
<%@page import="dataaccess.*"%>
<%@page import="valueobject.*"%>
<%@page import="session.title.*"%>

<!--
    Remember this JSP is for example purposes only.
    In a real-world application you should never allow a JSP to directly
    access the DAO.
-->
-->
<H1>Example 1 - Using a DAO to retrieve a title</H1>
<BR/>
<BR/>
<P>
    This JSP page will retrieve a single record from the Title table using
    a TitleDAO <BR/>
    and TitleVO object. It will also display all of the editors associated
    with this <BR/>
    Title by calling the getEditors() method on the TitleVO.
</P>
<BR/>
<%
    try {
```

The selectExample page retrieves a single record from the title table. It does this by retrieving a TitleDAO EJB:

```
ServiceLocator serviceLocator = ServiceLocator.getInstance();
TitleDAOHome titleDAOHome = (TitleDAOHome)
    serviceLocator.getEJBHome(ServiceLocator.TITLED AO);

TitleDAO titleDAO = titleDAOHome.create();
```

A TitleDAO EJB, called titleDAO, is retrieved by using a ServiceLocator to find the TitleDAO's EJBHome interface and then uses the EJBHome interface to retrieve the TitleDAO's remote interface.

Once a `TitleDAO` is retrieved, the `find()` method on the DAO is invoked, passing in the primary key of the record to be retrieved:

```
/*Calling the find method and retrieving a Title by title ID.*/
TitleVO titleVO = (TitleVO) titleDAO.findByPrimaryKey("1");
```

After the `TitleVO` is retrieved via the `TitleDAO.find()` method, it is used to print information about the title record requested:

```
out.println("<P>Title id    : " + titleVO.getTitleId()    + "<br/>");
out.println("<P>Title descr : " + titleVO.getTitleDescr() + "<br/>");
out.println("<P>Title cost  : " + titleVO.getTitleCost()  + "<br/>");

out.println("<P>Editor(s) Working on the Title : <br/>");
```

Remember, there is a many-to-many relationship between titles and editors. The editors associated with the title record retrieved can be accessed by calling the `getEditors()` method on `titleVO`:

```
/*
 * Retrieving the editors that the user currently has. We will
 * then grab an iterator off of the HashMap and then display the
 * names of each editor.
 */
HashMap hashMap = titleVO.getEditors();
```

Once a `HashMap` is retrieved from the `getEditors()` call, it is less work to cycle through the `HashMap` and retrieve an `EditorVO` for each editor record. Each retrieved `EditorVO` object can be queried for the editor's name:

```
Collection col = hashMap.values();

Iterator iterator = col.iterator();

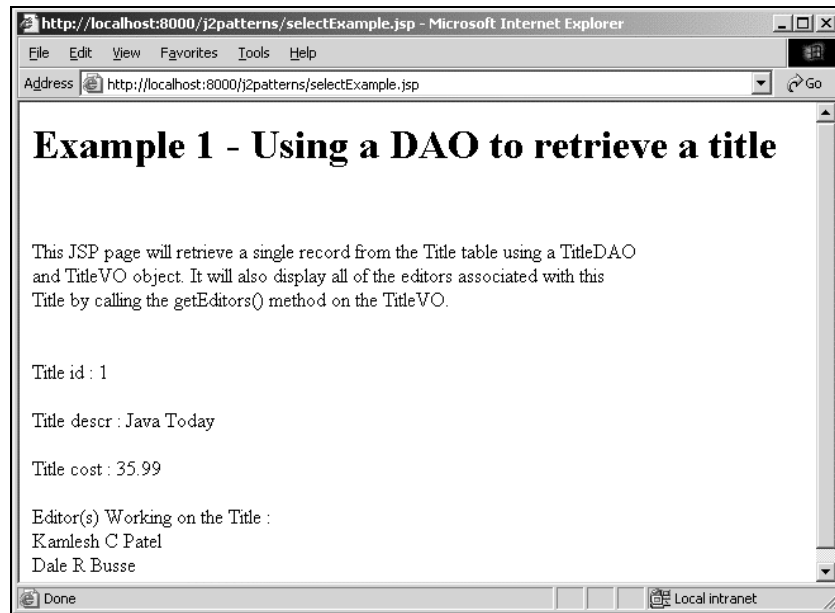
while (iterator.hasNext()) {
    EditorVO editorVO = (EditorVO) iterator.next();

    out.println(editorVO.getFirstName() + " " +
                 editorVO.getMiddleName() + " " +
                 editorVO.getLastName() + "<br/>");
}
} catch (Exception e) {
    out.println("error--> " + e.getMessage());
}
}

%>
```

The JSP page example used in this section is very simplistic, but it should drive home a simple point. Through the judicious use of persistence design patterns, we have been able to completely decouple any knowledge of the subscription database from the JSP page. The `selectExample.jsp` page has no idea where the title data is coming from or even how it is being retrieved.

When the `selectExample.jsp` page is run successfully, you should see the following results:



The `insertExample.jsp` Page

The `insertExample.jsp` page builds upon what was shown in the `selectExample.jsp` page. The `insertExample.jsp` will insert a new record into the title table using a `TitleDAO`. In the process of inserting this record, the `insertExample.jsp` page will also use the `EditorDAO` to retrieve editors that are then associated with the title record being inserted. The code for the `insertExample.jsp` page is shown below:

```

<%@page import="java.util.*"%>
<%@page import="javax.naming.*"%>
<%@page import="javax.transaction.*"%>
<%@page import="dataaccess.ServiceLocator"%>
<%@page import="valueobject.*"%>
<%@page import="session.title.*"%>
<%@page import="session.editor.*"%>

<!--
Remember this JSP is for example purposes only. In a real-world
Application you should never allow a JSP to directly access the DAO.
You would instead access the DAO from a session EJB.
  
```

```
-->

<H1>Example 2 - Using a DAO to insert a title</H1>
<BR/>
<BR/>
<P>
    This JSP page will insert a single record from the Title table using a
    TitleDAO <BR/>
    and TitleVO object.
    It will also establish relationships between the title <BR/>
    and the editor by retrieving 3 editorVO's and then adding them to the
    HashMap returned <BR/>
    by the getEditors() method on the TitleVO.<BR/>
</P>
<BR/>

<%
    try {
```

The first thing the `insertExample.jsp` page does is to retrieve a `TitleDAO` and `EditorDAO` from the application server. These two DAOs will be used to build the `TitleVO` object that is going to be inserted into the subscription database:

```
/*Getting a TitleDAO for manipulating title information*/
ServiceLocator serviceLocator = ServiceLocator.getInstance();
TitleDAOHome titleDAOHome = (TitleDAOHome)
    serviceLocator.getEJBHome(ServiceLocator.TITLEDAO);
TitleDAO titleDAO = titleDAOHome.create();

/*Getting an EditorDAO to retrieve Editor information*/
EditorDAOHome editorDAOHome = (EditorDAOHome)
    serviceLocator.getEJBHome(ServiceLocator.EDITORDAO);
EditorDAO editorDAO = editorDAOHome.create();
```

After the two DAOs are created, three editor records are retrieved from the subscription database (later, in the `insertExample.jsp` code, these records will be associated with the title record after it has been created):

```
/*
 * Looking up three editors. I am going to associate these editors
 * to the new title record
 */
EditorVO editorVO1 = (EditorVO) editorDAO.findByPrimaryKey("1");
EditorVO editorVO2 = (EditorVO) editorDAO.findByPrimaryKey("2");
EditorVO editorVO3 = (EditorVO) editorDAO.findByPrimaryKey("3");
```

Now, the real work begins. A new TitleVO object, called newTitleVO, will be created by calling the createValueObject() method on the titleDAO EJB:

```
/*Creating a new title vo by calling the TitleVO's
 * createValueObjectMethod
 */
TitleVO newTitleVO = (TitleVO) titleDAO.createValueObject();
```

The newTitleVO object is an empty TitleVO object. The only property that is set on this object is the titleId. This property holds a primary key value that was generated inside the createValueObject() method. If you remember from earlier discussions, the TitleDAO.createValueObject() method uses the Counter EJB to generate a primary key.

After the newTitleVO has been created, the JSP page populates it with data:

```
/*Populate the new TitleVO with data*/
newTitleVO.setTitleDescr("Geek World");
newTitleVO.setTitleCost(100);
```

The JSP page also associates the three EditorVOs retrieved earlier by the EditorDAO. Each EditorVO retrieved is added to a HashMap stored inside the newTitleVO object:

```
/*Add the editors I retrieved from the EditorDAO to the TitleVO*/
newTitleVO.getEditors().put(
    new Long(editorVO1.getEditorId()), editorVO1);
newTitleVO.getEditors().put(
    new Long(editorVO2.getEditorId()), editorVO2);
newTitleVO.getEditors().put(
    new Long(editorVO3.getEditorId()), editorVO3);
```

An entry in the title_editor table will be created for each of the EditorVOs added to the newTitleVO object:

```
/*Insert the record into the DAO*/
titleDAO.insert(newTitleVO);

/*Retrieving the record I just inserted into the database*/
TitleVO titleVO = (TitleVO) titleDAO.findByPrimaryKey(
    new Long(newTitleVO.getTitleId()).toString());

/*Printing out the TitleVO data just inserted*/
out.println("<B><P>Record just inserted:</P></B><BR/>");

out.println("<P>Title id    : " + titleVO.getTitleId()    + "<br/>");
out.println("<P>Title descr : " + titleVO.getTitleDescr() + "<br/>");
out.println("<P>Title cost  : " + titleVO.getTitleCost()  + "<br/>");

out.println("<P>Editor(s) Working on the Title : <br/>");
HashMap hashMap = titleVO.getEditors();

Collection col = hashMap.values();
```

```

        Iterator iterator = col.iterator();

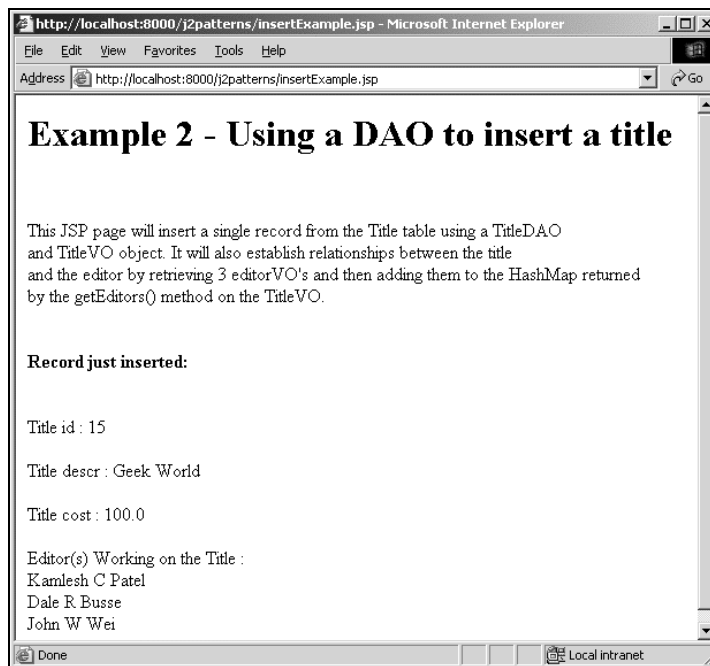
        while (iterator.hasNext()) {
            EditorVO editorVO = (EditorVO) iterator.next();

            out.println(editorVO.getFirstName() + " " +
                editorVO.getMiddleName() + " " +
                editorVO.getLastName() + "<br/>");
        }
    } catch (Exception e) {
        out.println("error--> " + e.getMessage());
    }
}
%>

```

Once the insertFlag for the newTitleVO object is set, the actual insertion of the record takes place by calling insert() on the titleDAO EJB. The rest of the code in the insertExample.jsp page retrieves the record (just inserted) back from the subscription database and displays it.

When the insertExample.jsp page runs successfully, you should see the following output on your web browser:



Persistence Framework Strategies

Providing a common interface for applications to work with databases within your organization is a powerful construct. However, in order to build a powerful persistence framework, you have to make sure that a number of issues are addressed. These issues include:

- ❑ Generating primary keys for records going into your database
- ❑ Handling concurrent updates by multiple users to the same database record in your application
- ❑ Identifying the best approach for transaction management within your persistence framework
- ❑ Optimizing the performance of your persistence framework
- ❑ Migrating your current Java data access code to a persistence framework

In the following sections, we will look at different strategies for addressing these issues.

Primary Key Generation Strategies

A primary key uniquely identifies a record with a database table. Primary keys are usually numeric values. Almost all databases provide some kind of mechanism for generating primary keys. From an application developer's perspective, there are a number of different approaches for generating primary keys for a database insert. Some of these approaches include:

- ❑ Using a database trigger to automatically generate a primary key when a record is inserted into the database. The trigger would use a sequence object (if the database supports it) to get the primary key.
- ❑ Using an auto-increment column construct to automatically insert a value when a record is inserted into the database. All leading vendors (for example, MySQL and Microsoft SQL*Server) offer auto-incrementing columns for generating primary keys.
- ❑ Directly using a database sequence generator to generate a primary key that can then be used for the insert into the database. An example of this would be Oracle's sequence object that can be queried for the next value in a sequence. This value can then be used when the insert occurs.
- ❑ Writing a stateful session EJB or entity EJB combination to maintain a cache of pre-generated sequences that can be invoked to retrieve a sequence number. The entity EJB would retrieve a sequence number from a table in the database and then update the counter value stored in the table. The session EJB would grab a block of these sequence numbers and cache them. The session EJB would then hand out these sequences to anyone requesting the EJB. When more sequence numbers are needed, the entity EJB would retrieve another set of numbers from the database. This way, the entity EJB is not constantly being hit with requests for sequences.

These four different approaches have their advantages and disadvantages. Using a database construct like a trigger, auto-increment column, or a sequence object is very easy to implement. Using these approaches ensures that the user never has to worry about generating a primary key. However, there are also some problems in using these approaches.

First, by using a database trigger to perform the primary key generation, there is no way of returning the primary key generated back to the code performing the database insert. This is a big problem if you need to establish a parent-child relationship in the data you are manipulating. For example, if your code is inserting an order and then order detail line items into the table, you will need the primary key of the order being inserted so that you can insert the order's detail line items.

Auto-incrementing database columns can offer the same challenges as a database trigger. How do you look up the primary key value after the insert? Before JDK1.3, this was a problem. However, with the release of JDK1.4, the `JDBC Statement` object now allows you to indicate whether you want an auto-generated value to be returned from an insert. However, since this feature has just been released in JDK1.4, check your JDBC driver documentation carefully to see if the driver supports this feature.

Database sequence objects are extremely powerful because they can be queried and the value can be returned before a database insert occurs. Oracle provides a sequence object that can be defined and then queried for primary key values. MySQL offers the `LAST_INSERT_ID()` function. Database sequences are very powerful, but they are also database specific. If database sequences are haphazardly scattered throughout the database code and not centralized into a single service, this can become a porting nightmare if your application has to be moved to a new database platform.

The last approach, the stateful session bean caching primary key values, is a popular implementation because it is very fast and can be written to be database independent. Cached primary keys require fewer calls to the database, which can offer a significant performance improvement in a high-transaction environment. However, there are a few things to keep in mind with this approach:

- ❑ The use of a stateful session EJB to cache primary key values can be very difficult to implement in a clustered environment. Each EJB in the cluster must ensure that the keys it has cached are not duplicated in any of the other EJBs.
- ❑ A stateful session EJB will lose all of the primary keys in its cache if the application server it is running in goes down. This can result in "gaps" in the primary keys stored in a database table. This is usually not a major problem, but for some database administrators, there is a level of discomfort when this situation occurs.
- ❑ There can be data corruption issues if there are other applications inserting data into the database that are not using the stateful session bean. Using database-specific constructs like triggers, auto-increment columns, and sequences is far less likely to encounter the problem.

What is the right approach? All of the approaches have been successfully implemented. Using database sequence objects is usually the easiest primary key generation mechanism. Since database sequences are database specific, it is important to write "wrapper" code that:

- ❑ Provides a single point of service for generating primary keys for your applications
- ❑ Decouples how primary keys are generated from the code actually using the primary keys
- ❑ Hides the database-specific features behind a façade

In the persistence framework presented in this chapter, the primary key generation was done via the `Counter EJB`. The `Counter EJB` is responsible for generating a unique number based on the counter ID passed into its `getNextVal()` method. The `Counter EJB` is used to generate both primary keys and row version numbers for the `TitleDAO` application.

The Counter EJB uses a combination of a database table to track sequences and the `LAST_INSERT_ID()` function of the MySQL database to generate primary keys in the subscription database. Let's walk through how the Counter EJB is implemented so that you can understand how to apply the concepts within this EJB to your own persistence framework.

The Counter EJB Database Structures

As there might be multiple counters in the subscription database, a table called counter was created. The counter table has two columns:

- ❑ `seq_num` – This column represents the current value for a counter
- ❑ `counter_name` – This column is the primary key for the counter table and represents a unique counter within the database

The Counter EJB is a standard session EJB with a home interface, a remote interface, and a bean class. The home and remote interfaces for the Counter EJB are shown below:

```
package session.counter;

import javax.ejb.EJBObject;
import java.rmi.RemoteException;
import dataaccess.DataAccessException;
import dataaccess.ServiceLocatorException;

public interface Counter extends EJBObject {
    public long getNextVal(String pCounterName)
        throws DataAccessException, ServiceLocatorException, RemoteException;
}
```

```
package session.counter;

import java.io.Serializable;
import javax.ejb.CreateException;
import java.rmi.RemoteException;
import javax.ejb.EJBHome;

public interface CounterHome extends EJBHome {
    Counter create() throws CreateException, RemoteException;
}
```

Now let's look at the bean implementation class:

```
package session.counter;

import javax.ejb.SessionBean;
import javax.ejb.SessionContext;
import java.sql.*;
import dataaccess.*;

public class CounterBean implements SessionBean{
    SessionContext ctx = null;
```

```
/*
 * The getNextVal method will retrieve the next sequence number
 * of the sequence the user requests. The sequence values are stored
 * in a database table called counter.
 */
public long getNextVal(String pCounterName)
    throws DataAccessException, ServiceLocatorException {

    long          valueReturned  = 0;
    Connection     conn          = null;
    PreparedStatement updateStatement = null;
    ResultSet      rsCounter     = null;

    try {
        //Getting a connection to the subscription database
        ServiceLocator serviceLocator = ServiceLocator.getInstance();
        conn = serviceLocator.getDBConn(ServiceLocator.SUBSCRIPTIONDB);
```

The process of retrieving a primary key starts when the user calls the `getNextVal()` method on the Counter EJB and passes in a `String` value representing the counter the application wants to increment.

The code below will update the sequence number for the counter being requested:

```
//Building a SQL statement that will update the counter
SQLCode sqlCode = SQLCode.getInstance();
String sql = sqlCode.getSQLStatement("counter.nextval.update");

//Executing the SQL statement
updateStatement = conn.prepareStatement(sql.toString());
updateStatement.setString(1, pCounterName);
updateStatement.execute();
```

The SQL code class is used to retrieve the SQL statement shown below:

```
UPDATE counter SET seq_num = LAST_INSERT_ID(seq_num+1)
WHERE counter_name = ?
```

You will notice that the `UPDATE` statement will call the MySQL `LAST_INSERT_ID()` function passing in the value of the current `seq_num` column, incremented by one. The `LAST_INSERT_ID()` function is behaving as a sequence generator, returning the expression passed into the function. The value returned by the `LAST_INSERT_ID()` function will be unique to each client issuing the above SQL statement. Since we are using a MySQL-specific function to implement the Counter EJB, we would have to rewrite the counter class if it were to run against a different database platform.

Once the update has completed, the code will immediately retrieve the incremented `seq_num` value by issuing a `SELECT` statement that invokes the `LAST_INSERT_ID()` function:

```
sql = sqlCode.getSQLStatement("counter.nextval.select");
```



```
//Executing the sql statement
updateStatement = conn.prepareStatement(sql.toString());
updateStatement.setString(1, pCounterName);
rsCounter = updateStatement.executeQuery();
```

The SQL code associated with the `counter.nextval.select` key is:

```
SELECT LAST_INSERT_ID() seq_num FROM counter WHERE counter_name = ?
```

Using the `LAST_INSERT_ID()` function in the `SELECT` statement above will cause the `seq_num` value that was updated in the `UPDATE` call earlier to be returned. The value retrieved from the `SELECT` statement is returned to the application calling the `getNextVal()` method:

```
//Pulling the new sequence id
if (rsCounter.next()){
    valueReturned = rsCounter.getLong("seq_num");
}
} catch(SQLException e) {
    //Marking the session bean for rollback
    ctx.setRollbackOnly();
    throw new DataAccessException(
        "Error has occurred in CounterBean.getNextVal()", e);
} finally {
    try {
        //Close my database connections
        if (updateStatement != null) updateStatement.close();
        if (rsCounter!=null) rsCounter.close();
        if (conn!=null) conn.close();
    } catch(SQLException e) {
        System.out.println("Unable to close resultset, database " +
            "connection or statement in Counter.getNextVal()");
    }
}
return valueReturned;
}

public void ejbCreate(){}
public void ejbRemove() {}
public void ejbActivate() {}
public void ejbPassivate() {}
public void setSessionContext(SessionContext pCtx){
    ctx = pCtx;
}
}
```

Remember, this is only one way of implementing a primary key generation strategy. The power of this approach comes from the fact that you can completely rewrite how primary keys are generated within your application and the DAOs using the primary key generator will never have to be rewritten.

Concurrency Strategies

Imagine the following scenario: Two users are trying to update the same record. They both have retrieved copies of the record and User 1 submits some changes and commits. User 2 then submits some changes on the same record and commits. User 2's changes have now overwritten the data just saved by User 1.

The problem here is that the two users' transactions are going on concurrently. Neither user knows that they have both retrieved and are working on copies of the same record. Furthermore, User 1 will never know that User 2 made a change to record they just committed. What is needed is a locking strategy that will prevent one user from overwriting another user's record when both users are working on the same record concurrently. Two different strategies can be applied here:

- ❑ The first user to retrieve the record can explicitly place a database lock on the record and prevent all other users from retrieving or manipulating that record until they have explicitly released the lock. This strategy is called a **pessimistic locking strategy** because it says no one can touch the database record, even if they only want to read it, until the first user is done with. Pessimistic locking is the safest way to isolate one user's transactions from another. However, it is a performance killer in a high transaction volume application because one user locking a record can quickly cause other users to have to wait. As each user has to wait, they may be locking other records other users might need and you can quickly end up in a deadlock.
- ❑ The second strategy is to allow both users to retrieve a copy of the record. Their retrieved copy will include some kind of version information. This version information on the retrieved record will be compared against the version number stored in the database. If the version information indicates that an update has not occurred on a record a user is trying to update, then the update can occur. This strategy is called optimistic locking. Each user is allowed to access data and make their modifications to data without the overhead of placing an explicit database lock.

Optimistic locking is best suited for high transaction environments where a small amount of data is going to be retrieved and then quickly updated.

As stated earlier, the persistence framework built in this chapter uses an optimistic locking strategy. Optimistic locking strategies can be very easy or very difficult to implement depending on whether the optimistic locking approach is only going to be used for new application development, or will be used for existing applications.

There are a number of different ways of implementing an optimistic locking strategy for a brand new application. Two of the more common mechanisms for implementing an optimistic locking strategy are:

- ❑ Having a row version column on each row in a table. Before a record is updated, the row version of the record being updated is compared with the row version of the record in the database.
- ❑ Having a date/timestamp column that holds the timestamp of when a record was last updated. Before an update is allowed to occur, the date/timestamp of the record in the database and the date/timestamp of record that is going to be used for the update are compared. If the date/timestamp on the record in the database is newer than the record being used for the update, then record being used for the update is considered stale.

For the subscription database, a `row_version` number-based optimistic locking strategy was implemented. The `row_version` optimistic locking strategy requires that a `row_version` column be added to each table in a database. This `row_version` column will be updated on a specific record every time the record is updated. The actual checking for whether or not data being updated is "stale" can be implemented in one of two ways:

- ❑ Put the code that checks to see if there is stale data for an update inside your DAO layer. This was demonstrated in the `update()` method of the `TitleDAO` EJB earlier in the chapter. The `update()` method attempts to update a record in the `title` table by using the primary key and the row version in the `WHERE` clause of the `UPDATE` statement.
- ❑ Use database triggers to implement your optimistic locking strategy. Each table in your database has a database trigger that will fire whenever an update on a record occurs. The trigger compares the row version of the record being updated against the record currently in the database. If they are the same, the record is updated. If the row version numbers differ, the trigger raises an exception and the record is not updated.

Using database triggers to manage an optimistic locking strategy is actually the easier of the two methods to implement. You do not have to clutter up your DAOs with optimistic locking code. All of this is done at the trigger level. Furthermore, you do not have to worry whether all applications that update data in your application are also implementing optimistic locking code.

The disadvantage of using the trigger-based approach is that database triggers can be very database specific and this makes it more difficult to support your application across multiple databases. However, a well-designed case tool, like ER-WIN, can help mitigate this by providing the ability to generate triggers for multiple databases through the tool. This simplifies things tremendously.

For this chapter, the optimistic locking code was put inside of the DAO layer simply because the MySQL database does not support triggers.

For existing applications, implementing an optimistic locking strategy can be a significant amount of work. If you have ownership of the application source code, implementing an optimistic locking strategy often involves walking through the entire code base and changing all database selects and updates to implement the optimistic locking strategy. Furthermore, the application code must be changed to handle any optimistic locking exceptions raised during the course of a transaction. This is not an easy task.

If you do not own the application source code, as is often the case with products bought from third-party vendors, you will need to work with the vendor to see if an optimistic locking strategy can be implemented. If the vendor is not willing to implement one, you are in a bind. However, if you are currently in the process of selecting a package that you know you are going to have to integrate with other applications within your organization, ask how they are implementing a locking strategy for the application in question.

If the vendor is using pessimistic locking, locking a database resource via issuing a lock on a row or table in the database can lead to performance problems in a high transaction environment. If they are not using any kind of locking strategy (you would be surprised at the number of application vendors who do not), this can lead to "corrupt" data when you need to integrate that package with applications in your organization. Asking questions like this, at the package selection phase, can save a significant amount of headaches later on when you are trying to perform application integration.

Transaction Management Strategies

J2EE offers two different approaches for managing database transactions: bean-managed or container-managed demarcation. For bean-managed transactions, the application developer must place transaction code in their DAO (assuming the JDBC code is placed directly inside the DAO) or entity bean (if bean-managed persistence entity beans are used). The developer is responsible for beginning the transaction and then committing or rolling back the transaction.

The developer controls the transaction by using the Java Transaction API (JTA). If the developer forgets to commit or rollback a transaction before leaving a method call on a bean-managed transaction EJB, an exception will be thrown by the EJB container. The exact exception thrown is specific to the container the EJB is running in.

In container-managed transactions, also called declarative transaction management, the container the EJB is running in manages the transaction. The application developer declares in the EJB's deployment descriptor how the container is supposed to manage transactions. There are six different transaction types that can be declared inside an EJB's deployment descriptor:

- ❑ **NotSupported**
With the `NotSupported` transaction type, the EJB in question cannot be involved in any transactions. If an EJB starts a transaction and then calls another EJB with a transaction type of `NotSupported`, then the calling EJB will have its transaction suspended. Since the DAO pattern is used for handling database transactions, this transaction type should never be used for declaring a DAO EJB.
- ❑ **Supports**
The `Supports` transaction type tells the container to run the EJB in the transaction if a transaction has already been started. If a transaction has not been started, the EJB will not run in a transaction. Since a DAO can insert, update, or delete database records it should always run in a transaction. Since `Supports` transaction type does not guarantee that an EJB will be run in a transaction it is highly recommend you do not use it for building the DAO EJBs seen in this chapter.
- ❑ **Required**
The `Required` transaction type forces the EJB to run within a transaction. If a transaction already exists, the EJB will run in the context of that transaction. If a transaction does not exist, a new one will be started and the EJB will run in that transaction.
- ❑ **RequiresNew**
The `RequiresNew` transaction causes the EJB to run in a transaction independent of any transactions currently running. This transaction type is used when you want to run database transactions independently of one another.
- ❑ **Mandatory**
The `Mandatory` transaction type requires that the EJB run in the context of an already running transaction. If no transaction is already running, then a `javax.ejb.TransactionRequiredException` is thrown back to the calling application.

- ❑ **Never**
The `Never` transaction type tells the EJB container that the EJB should never run in a transaction. If it is called by code that is currently running within a transaction, an exception will be thrown. The exception thrown will vary depending on whether or not the EJB is called as a local EJB or a remote EJB. The exception thrown will be `javax.ejb.EJBException` or `javax.ejb.RemoteException` respectively.

In terms of writing a data-persistence framework for your applications, it is highly recommended that you use container-managed transactions. There are several reasons for this:

- ❑ A persistence framework written with container-managed transactions is significantly easier. All transaction management is done declaratively and you do not have to clutter up your DAO code with transaction code. Transaction code can be extremely difficult to write and debug when you have to write transactions that span multiple components.
- ❑ Container-managed transactions are more flexible. If you want to change the transaction behavior of an EJB, it is a simple matter of changing the EJB's deployment descriptor and re-deploying the bean. For instance, suppose you wanted to ensure that commits made by the `Counter` EJB always run in a new transaction. If the `Counter` EJB is currently set to transaction type of `Required`, it is a simple matter to change the transaction type in `Counter`'s deployment descriptor to be `RequiresNew` and then redeploy the EJB. No changes have to be made to the `Counter` code for this to take place.
- ❑ Poorly written transaction code can cripple the performance of an application. Remember, that while a transaction is in progress, application server and database resources are used to track the progress of the transaction. Too often, developers writing bean-managed transaction code keep a transaction open for longer than needed. In high-volume, transaction-processing applications, like most e-commerce web sites, poor coding of transaction management can result in abysmal performance.

Keep in mind that bean-managed persistence offers a high degree of flexibility and ultimately gives the persistence framework developer the highest degree of transaction control and isolation that can be obtained. However, this level of control is rarely needed for most applications. This section only gives a brief overview of EJB transaction management.

Performance Strategies

Abstraction and encapsulation all come at the cost of performance. Building a data-persistence framework will rarely be as high performance as going directly after the data. However, the reuse and the lower cost of maintenance associated with implementing a data-persistence framework often outweighs the performance costs associated with it.

However, there are some very common mistakes when implementing data-persistence patterns that can severely limit the performance and scalability of these applications. Some of the more common performance issues can range from design errors to a misunderstanding of how EJB's function:

- ❑ The wrong level of granularity for the DAOs in your persistence framework
- ❑ A large number of value objects returned from DAO `find()` method calls
- ❑ Common EJB implementation pitfalls

Persistence framework developers often fall into the trap of trying to map their data model directly to DAO classes. The result is a data-persistence framework that has too many DAO classes and retrieves large numbers of fine-grained value objects.

A persistence framework should be built on the concept of coarse-grained objects. The DAO in your persistence framework should not be returning large numbers of value objects. So when designing your data-persistence framework keeps in mind the following:

- ❑ **Watch the breadth of the objects being returned by the DAOs in your framework**
Returning too many value objects from a call on a `find()` method will result in your application quickly running into Java Virtual Machine (JVM) memory issues. Persistence frameworks that do not have the right level of granularity will have problems with frequent garbage collection or running out of memory in the JVM.
- ❑ **Watch the depth of the objects being returned by the DAOs in your framework**
A common mistake in applying persistence patterns is making the object hierarchy for your value objects too deep. This too is a granularity issue. Every value object returned should not have more than three levels of other objects contained within it. Otherwise, even the act of retrieving or updating a few value objects can represent a significant amount of overhead.

The persistence patterns presented in this chapter work well when writing OLTP (Online Transaction Processing) applications where your application is working with a relatively small data set that is going to be manipulated by an end user. However, you can run into serious scalability and performance issues implementing these patterns in an OLAP (Online Analytical Processing) environment like a reporting database or a data warehouse.

In these situations, you are far better off going directly after the data using JDBC to directly retrieve a `ResultSet`. In order to get a `ResultSet`, the application must get a JDBC connection and then issue a SQL query and retrieve a `ResultSet`. In doing this, it would appear that every abstraction principle the persistence framework works so hard to establish is now violated. However, architecture is never a pure thing and occasional exceptions must be made.

However, there are some more choices provided by the Optional Package (and now in JDK 1.4):

The JDBC Optional Package (found in J2EE) introduced the concept of a `RowSet`. A `RowSet` extends the JDBC `ResultSet` with one important difference. A `RowSet`, can be disconnected from the database connection, so even if the JDBC connection is closed, the `RowSet` can still be used to read and manipulate data. Later the `RowSet` can re-establish a connection to the database and update it with any changes that were made to the data contained within it.

If you only need to retrieve large amounts of data infrequently, you can integrate and add a `find()` method to your DAO that returns a `RowSet`. Remember, a DAO pattern will abstract away how data is actually retrieved. This means you can have one `find()` method returning a `RowSet` using JDBC and another `find()` method returning a value object that was read from an entity EJB. For bulk updates, inserts, or deletes you can even pass the `RowSet` back into the DAO and have DAO perform the updates, inserts or deletes on behalf of the application using the `RowSet`.

Another common performance issue involved with implementing a J2EE-based persistence framework is that the framework developer falls into common pitfalls associated with writing EJB-based applications. Since persistence frameworks often involve many small transactions executed against a database, there are two things to keep in mind:

- ❑ Be aware that every time a JNDI lookup is performed, there is overhead associated with it. If you have a situation where one heavily used DAO is using several other DAOs to carry out a transaction, consider using a Service Locator with a caching strategy, such as the one demonstrated in this chapter, to cut down on the number of JNDI lookups performed. In a high transaction environment, this can significantly cut down on the number of JNDI lookups being performed.
- ❑ Use local and remote EJB interfaces appropriately. If your DAO layer is only going to be used by application code running within your application server, consider using local interfaces for your DAO implementation. Local interfaces are part of the EJB 2.0 specification and significantly reduce the amount of network traffic that has traditionally been the bane of any EJB application development.

Always, design your persistence framework with performance in mind from the beginning. It is much easier to implement the performance strategies discussed above while the framework is still being developed and the code is fluid than after several applications have been built on top of the framework:

Good architecture starts from the beginning and not as an afterthought when a project has been deployed.

Migration Strategies

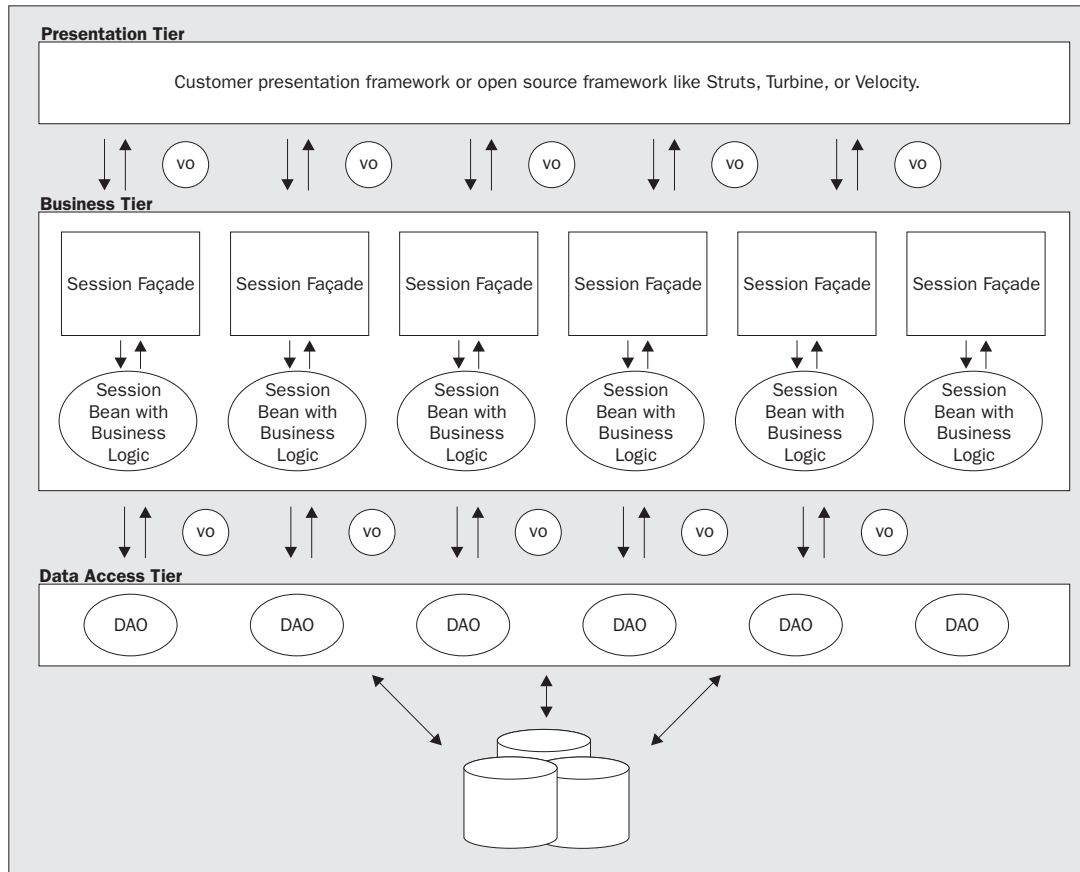
Most IT developers would love to spend their careers writing applications from scratch. However, the reality is that the majority of a developer's time is spent supporting existing code for integrating applications together.

Persistence frameworks are always easiest to implement for new applications. How do you implement a persistence framework for J2EE applications that have already been written? The answer: start small and slow. Most J2EE applications have leveraged session EJBs to build the business logic for their applications. However, they have intermixed data access code all through their session EJB layer.

The key is to start small and identify a few key data entities that you want represented in a persistence layer. Usually, the entities that work best are going to be data entities that are used throughout all of the applications in your organization. For most companies, customer data is the most critical piece of information in their databases. Take that data entity and use that as the foundation for building your persistence framework.

Before you begin any implementation of the persistence framework, whether the implementation is for new application development or a migration of existing applications, it is critical that you have a conceptual road map to build your entire framework. This road map is a diagram that lays out all of the tiers in your framework and how the different pieces are going to interact together.

Shown below is an example of a framework road map:



The framework shown above is n-tiered architecture using a closed-layer approach. It is a closed-layer approach because each tier can only talk to the tiers directly above and below it. The presentation tier can only talk to the business tier, the business tier can only talk to the presentation and data tiers, and the data tier is the sole point of access to all corporate data sources.

The business tier in the above road map is split into two pieces: Session Façades and session beans. The Session Façades are Java classes that sit between the presentation tier and the session EJBs. Session Façades wrap session EJBs and keep the presentation layer from being exposed to which EJBs are being used to process business logic. The session beans are using the DAO in the data access tier to retrieve and manipulate data. Value objects are the only pieces of the architecture that can move data back and forth between the tiers.

Once the conceptual road map for your framework has been established and you understand how your applications are going to communicate with the data tier, you can begin the process of migrating your existing application base over to a persistence framework.

Some of the steps involved with this migration include:

- ❑ Look through all of the applications that are currently using customer data. Identify core data elements of the customer that are repeatedly used throughout the applications. Also, identify how different business processes are using these data elements.
- ❑ Build the base classes of your persistence framework. Identify the data sources you are going to need and wrap them in a `DataSourceFactory`.
- ❑ Identify how primary keys in your framework are to be generated. Look at the underlying databases you are going to be accessing and develop a centralized mechanism for generating primary keys that is consistent with how customer records are currently stored.
- ❑ Identify whether or not you need to implement an optimistic locking mechanism. Remember, if you start small, you do not need to implement optimistic locking on every table in your database.
- ❑ Build the DAO and VO classes that you are going to use. If the majority of the code is written as JDBC code, look at starting out with using JDBC directly in your session EJB.
- ❑ Walk through each of the applications that use the customer data and convert them over to the new persistence framework.

It is highly recommended that you do not go for a big bang with your migration strategy. Your migration strategy should start with a small number of data entities that will be built and no radical shifts in database access technology.

This means that if your application code uses JDBC, then stick with that during your persistence framework implementation. Do not jump to container-managed persistence or some other radically different approach. That can come later in other iterations of your persistence framework. Otherwise, you are going to find yourself not only debugging the patterns you are implementing, but also trying to support a new data access technology your team might not be familiar with.

Summary

The J2EE platform offers a flexible platform for building a persistence framework. Its component-based architecture and multiple database APIs provide readymade tools for crafting a persistence framework that can be leveraged for most of, if not all, your application development needs.

Data is the only thing that remains consistent within an organization's IT infrastructure. Applications come and go. However, the data used by them rarely disappears. It might be migrated to another database platform or transformed to a different format or structure, but it is still fundamentally the same. After all, when an organization implements a new Customer Relationship Management application does it throw out all of its customer data? No, it migrates that data to the new system, often massaging it into a new data structure.

Herein lies the power of a data-persistence framework. All application development frameworks provide a degree of code reuse. However, persistence frameworks go beyond letting developers reuse code. A data-persistence framework defines a standard set of interfaces for retrieving and manipulating data entities that are important to your organization.

The underlying technology of the persistence framework might change, but you get reusability because not all of the applications using the framework have to change when those data entities migrate or move. Only the implementation of the framework changes, not the public interfaces the applications uses to retrieve and manipulate data:

The power of a persistence framework comes from its ability to abstract away the underlying data source implementation.

This chapter has provided the three basic patterns that lay a foundation for a data-persistence framework. The patterns are:

- ❑ The Data Access Object pattern that hides how data for a particular business data entity is retrieved, inserted, updated, and deleted
- ❑ The Value Object pattern that abstracts away the physical relationships that exist between data entities residing in a corporate data source
- ❑ The Service Locator pattern that abstracts away how EJB and JDBC connections are retrieved from the database

In addition to these core patterns, this chapter also provided a number of strategies for dealing with such issues as:

- ❑ Primary key generation
- ❑ Concurrency management
- ❑ Transaction management
- ❑ Performance strategy
- ❑ Migration strategies for moving your existing applications to a persistence framework

